

Soil FW Placa de Controle

v2.0.0

Generated by Doxygen 1.9.1

1 Sample project	1
1.1 How to use example	1
1.2 Example folder contents	1
1.3 ESP IDF	1
1.4 Compilation	2
1.5 Flash	2
2 Module Index	3
2.1 Modules	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	7
4.1 File List	7
5 Module Documentation	9
5.1 FreeRTOS Defines	9
5.1.1 Detailed Description	11
5.2 Project Configuration	11
5.2.1 Detailed Description	12
5.2.2 Macro Definition Documentation	12
5.2.2.1 CONFIG_HISTORY_MAX_VALUE	12
5.2.2.2 CONFIG_SCHEDULING_MAX_VALUE	12
5.2.2.3 CONFIG_SECTORS_MAX_VALUE	13
5.2.3 Typedef Documentation	13
5.2.3.1 app_callback	13
5.2.4 Enumeration Type Documentation	13
5.2.4.1 comm_type	13
5.2.4.2 idp_type	13
5.2.4.3 pivot_states	14
5.2.5 Function Documentation	14
5.2.5.1 __attribute__()	14
5.2.6 Variable Documentation	15
5.2.6.1 global_angle	15
6 Data Structure Documentation	17
6.1 arg_pair Struct Reference	17
6.1.1 Detailed Description	17
6.1.2 Field Documentation	17
6.1.2.1 type	17
6.1.2.2 value	17
6.2 async_resp_arg Struct Reference	18
6.3 http_file_server_data Struct Reference	18

6.3.1 Detailed Description	18
6.4 i2c_dev_t Struct Reference	18
6.4.1 Detailed Description	18
6.5 pivot_sectors Struct Reference	18
6.5.1 Detailed Description	19
6.5.2 Field Documentation	19
6.5.2.1 end_angle	19
6.5.2.2 start_angle	19
6.6 rtc_i2c_dev_t Struct Reference	19
7 File Documentation	21
7.1 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/actuation_↔ app.c File Reference	21
7.1.1 Detailed Description	22
7.1.2 Function Documentation	22
7.1.2.1 actuation_app_get_actions()	22
7.1.2.2 actuation_app_init()	23
7.1.2.3 actuation_app_manual_call()	23
7.1.2.4 actuation_app_set_actions()	23
7.1.2.5 actuation_app_set_config()	24
7.1.2.6 actuation_app_set_pump()	24
7.1.2.7 actuation_app_shutdown()	24
7.1.2.8 actuation_app_task()	24
7.1.3 Variable Documentation	25
7.1.3.1 pivot_actions_off	25
7.2 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/comm_app.c File Reference	25
7.2.1 Detailed Description	26
7.2.2 Function Documentation	26
7.2.2.1 comm_app_init()	26
7.2.2.2 comm_app_send_idp_pack()	27
7.2.2.3 comm_app_wifi_config()	27
7.3 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/data_app.c File Reference	27
7.3.1 Detailed Description	29
7.3.2 Function Documentation	29
7.3.2.1 data_app_delete_scheduling()	29
7.3.2.2 data_app_gen_scheduling_key()	29
7.3.2.3 data_app_get_data_size()	30
7.3.2.4 data_app_init()	30
7.3.2.5 data_app_load()	30
7.3.2.6 data_app_save()	31

7.4 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/actuation↵ _app.h File Reference	31
7.4.1 Detailed Description	33
7.4.2 Function Documentation	33
7.4.2.1 actuation_app_get_actions()	33
7.4.2.2 actuation_app_init()	33
7.4.2.3 actuation_app_set_actions()	34
7.4.2.4 actuation_app_set_config()	34
7.4.2.5 actuation_app_set_pump()	35
7.4.2.6 actuation_app_shutdown()	35
7.4.3 Variable Documentation	36
7.4.3.1 pivot_actions_off	36
7.5 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/comm↵ _app.h File Reference	36
7.5.1 Detailed Description	37
7.5.2 Function Documentation	37
7.5.2.1 comm_app_init()	37
7.5.2.2 comm_app_send_idp_pack()	38
7.5.2.3 comm_app_wifi_config()	38
7.6 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/data↵ _app.h File Reference	39
7.6.1 Detailed Description	40
7.6.2 Enumeration Type Documentation	40
7.6.2.1 data_type_t	40
7.6.3 Function Documentation	41
7.6.3.1 data_app_delete_scheduling()	41
7.6.3.2 data_app_gen_scheduling_key()	41
7.6.3.3 data_app_get_data_size()	42
7.6.3.4 data_app_init()	42
7.6.3.5 data_app_load()	43
7.6.3.6 data_app_save()	43
7.7 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/rtc↵ _app.h File Reference	44
7.7.1 Detailed Description	45
7.7.2 Function Documentation	45
7.7.2.1 rtc_app_get_date_time()	45
7.7.2.2 rtc_app_get_str_date_time()	46
7.7.2.3 rtc_app_get_timestamp()	46
7.7.2.4 rtc_app_init()	46
7.7.2.5 rtc_app_set_timestamp()	47
7.7.2.6 rtc_show_date_time()	47
7.8 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/rtc_app.c File Reference	47

7.8.1 Detailed Description	48
7.8.2 Function Documentation	49
7.8.2.1 rtc_app_get_date_time()	49
7.8.2.2 rtc_app_get_str_date_time()	49
7.8.2.3 rtc_app_get_timestamp()	49
7.8.2.4 rtc_app_init()	50
7.8.2.5 rtc_app_set_timestamp()	50
7.8.2.6 rtc_show_date_time()	50
7.9 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gpio_actuator/gpio_↵ actuator.c File Reference	51
7.9.1 Detailed Description	52
7.9.2 Function Documentation	52
7.9.2.1 actuator_read_percent()	52
7.9.2.2 actuator_wait_pressure()	52
7.9.2.3 gpio_actuator_config()	53
7.9.2.4 gpio_actuator_get()	53
7.9.2.5 gpio_actuator_init()	54
7.9.2.6 gpio_actuator_pressure_off()	54
7.9.2.7 gpio_actuator_pressure_on()	54
7.9.2.8 gpio_actuator_pump_off()	54
7.9.2.9 gpio_actuator_pump_on()	54
7.9.2.10 gpio_actuator_set()	54
7.9.2.11 gpio_actuator_shutdown()	55
7.9.2.12 gpio_actuator_start()	55
7.9.2.13 vPercTimerOffExpire()	55
7.9.2.14 vPercTimerOnExpire()	56
7.10 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gpio_actuator/include/gpio_↵ _actuator.h File Reference	56
7.10.1 Detailed Description	58
7.10.2 Macro Definition Documentation	58
7.10.2.1 GPIO_ACT_PIN_AUX	58
7.10.2.2 GPIO_ACT_PIN_CCW	59
7.10.2.3 GPIO_ACT_PIN_CCW_IN	59
7.10.2.4 GPIO_ACT_PIN_CW	59
7.10.2.5 GPIO_ACT_PIN_CW_IN	59
7.10.2.6 GPIO_ACT_PIN_OFF	59
7.10.2.7 GPIO_ACT_PIN_ON	59
7.10.2.8 GPIO_ACT_PIN_PERC_AUX	60
7.10.2.9 GPIO_ACT_PIN_PERC_IN	60
7.10.2.10 GPIO_ACT_PIN_PERC_OUT	60
7.10.2.11 GPIO_ACT_PIN_PRESS	60
7.10.2.12 GPIO_ACT_PIN_PUMP	60
7.10.2.13 GPIO_ACT_PIN_WATERING	60

7.10.2.14	GPIO_INPUT_PIN_GROUP	61
7.10.2.15	GPIO_OUTPUT_PIN_GROUP	61
7.10.3	Function Documentation	61
7.10.3.1	gpio_actuator_config()	61
7.10.3.2	gpio_actuator_get()	62
7.10.3.3	gpio_actuator_init()	62
7.10.3.4	gpio_actuator_pressure_off()	63
7.10.3.5	gpio_actuator_pressure_on()	63
7.10.3.6	gpio_actuator_pump_off()	63
7.10.3.7	gpio_actuator_pump_on()	63
7.10.3.8	gpio_actuator_set()	63
7.10.3.9	gpio_actuator_shutdown()	64
7.11	/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gprs/include/gprs_uart.h File Reference	64
7.11.1	Detailed Description	65
7.11.2	Function Documentation	66
7.11.2.1	gprs_uart_init()	66
7.11.2.2	gprs_uart_send_event()	66
7.12	/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/idp_parser.c File Reference	67
7.12.1	Detailed Description	68
7.12.2	Function Documentation	68
7.12.2.1	idp_parser_create_package()	68
7.12.2.2	idp_parser_create_pwd()	68
7.12.2.3	idp_parser_get()	69
7.12.2.4	idp_parser_get_packet_data()	69
7.12.2.5	idp_parser_get_pwd()	70
7.12.2.6	idp_parser_validate_actions()	70
7.12.2.7	idp_parser_validate_network()	70
7.13	/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/include/idp_parser.h File Reference	71
7.13.1	Detailed Description	72
7.13.2	Function Documentation	72
7.13.2.1	idp_parser_create_package()	72
7.13.2.2	idp_parser_create_pwd()	73
7.13.2.3	idp_parser_get()	74
7.13.2.4	idp_parser_get_packet_data()	74
7.13.2.5	idp_parser_get_pwd()	75
7.13.2.6	idp_parser_validate_actions()	75
7.13.2.7	idp_parser_validate_network()	76
7.14	/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/nvs_data/include/nvs_data.h File Reference	77
7.14.1	Detailed Description	78

7.14.2 Function Documentation	78
7.14.2.1 nvs_data_get_blob()	78
7.14.2.2 nvs_data_get_size()	78
7.14.2.3 nvs_data_init()	79
7.14.2.4 nvs_data_set()	79
7.15 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/nvs_data/nvs_data.c File Reference	80
7.15.1 Detailed Description	81
7.15.2 Function Documentation	81
7.15.2.1 nvs_data_get_blob()	81
7.15.2.2 nvs_data_get_size()	81
7.15.2.3 nvs_data_init()	82
7.15.2.4 nvs_data_set()	82
7.16 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rf_module/include/rf_↔uart.h File Reference	82
7.16.1 Detailed Description	83
7.16.2 Function Documentation	84
7.16.2.1 rf_uart_init()	84
7.16.2.2 rf_uart_send_event()	84
7.17 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rf_module/rf_uart.c File Reference	85
7.17.1 Detailed Description	86
7.17.2 Function Documentation	86
7.17.2.1 rf_uart_init()	86
7.17.2.2 rf_uart_send_event()	86
7.18 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/include/rtc_ds3231.h File Reference	87
7.18.1 Detailed Description	89
7.18.2 Function Documentation	89
7.18.2.1 bcd2dec()	90
7.18.2.2 dec2bcd()	90
7.18.2.3 ds3231_get_raw_temp()	91
7.18.2.4 ds3231_get_temp_float()	91
7.18.2.5 ds3231_get_temp_integer()	92
7.18.2.6 ds3231_get_time()	93
7.18.2.7 ds3231_init_desc()	93
7.18.2.8 ds3231_set_time()	94
7.19 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/rtc_ds3231.c File Reference	95
7.19.1 Detailed Description	96
7.19.2 Macro Definition Documentation	96
7.19.2.1 CHECK_ARG	96
7.19.3 Function Documentation	96

7.19.3.1 bcd2dec()	97
7.19.3.2 dec2bcd()	97
7.19.3.3 ds3231_get_raw_temp()	97
7.19.3.4 ds3231_get_temp_float()	98
7.19.3.5 ds3231_get_temp_integer()	98
7.19.3.6 ds3231_get_time()	99
7.19.3.7 ds3231_init_desc()	99
7.19.3.8 ds3231_set_time()	100
7.20 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/rtc_i2c_dev.c File Reference	100
7.20.1 Detailed Description	101
7.20.2 Function Documentation	101
7.20.2.1 i2c_dev_read()	101
7.20.2.2 i2c_dev_write()	102
7.20.2.3 i2c_master_init()	102
7.21 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/FreeRTOS_defines.h File Reference	103
7.21.1 Detailed Description	105
7.22 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/log.h File Reference	105
7.22.1 Detailed Description	106
7.22.2 Macro Definition Documentation	106
7.22.2.1 LOG_ACTUATION	106
7.22.2.2 LOG_COLOR_WHITE	106
7.22.2.3 LOG_COMM	107
7.22.2.4 LOG_DATA	107
7.22.2.5 LOG_MANAGER	107
7.23 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/project_config.h File Reference	108
7.23.1 Detailed Description	110
7.24 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/wifi_app/include/wifi_app.h File Reference	110
7.24.1 Detailed Description	111
7.24.2 Function Documentation	111
7.24.2.1 wifi_app_init()	111
7.24.2.2 wifi_app_set_config()	112
7.25 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/wifi_app/wifi_app.c File Reference	112
7.25.1 Detailed Description	113
7.25.2 Function Documentation	113
7.25.2.1 wifi_app_init()	113
7.25.2.2 wifi_app_set_config()	113
7.25.2.3 wifi_app_start()	114
7.26 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/eco_mode.c File Reference	114

7.26.1 Detailed Description	115
7.26.2 Function Documentation	115
7.26.2.1 eco_mode_register_callback()	115
7.26.2.2 eco_mode_start()	115
7.27 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/eco_mode.h File Reference	116
7.27.1 Detailed Description	117
7.27.2 Function Documentation	117
7.27.2.1 eco_mode_register_callback()	117
7.27.2.2 eco_mode_start()	117
7.28 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/scheduling.h File Reference	118
7.28.1 Detailed Description	119
7.28.2 Function Documentation	119
7.28.2.1 scheduling_register_callback()	119
7.28.2.2 scheduling_start()	119
7.29 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/sectorization.h File Reference	120
7.29.1 Detailed Description	121
7.29.2 Function Documentation	121
7.29.2.1 sectorization_start()	121
7.29.2.2 sectorization_stop()	121
7.30 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/system_manager.h File Reference	122
7.30.1 Detailed Description	122
7.30.2 Function Documentation	122
7.30.2.1 system_manager_init()	122
7.31 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/system_monitoring.h File Reference	123
7.31.1 Detailed Description	124
7.31.2 Function Documentation	124
7.31.2.1 system_monitoring_register_callback()	124
7.31.2.2 system_monitoring_start()	124
7.31.2.3 system_monitoring_stop()	125
7.32 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/main.c File Reference	125
7.32.1 Detailed Description	126
7.32.2 Function Documentation	126
7.32.2.1 app_main()	126
7.33 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/scheduling.c File Reference	126
7.33.1 Detailed Description	127
7.33.2 Function Documentation	127
7.33.2.1 scheduling_register_callback()	127
7.33.2.2 scheduling_start()	128

7.34 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/sectorization.c File Reference	128
7.34.1 Detailed Description	129
7.34.2 Function Documentation	129
7.34.2.1 sectorization_start()	129
7.34.2.2 sectorization_stop()	129
7.35 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/system_manager.c File Reference	129
7.35.1 Detailed Description	130
7.35.2 Function Documentation	131
7.35.2.1 system_manager_init()	131
7.36 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/system_monitoring.c File Reference	131
7.36.1 Detailed Description	132
7.36.2 Enumeration Type Documentation	132
7.36.2.1 system_monitoring_states	132
7.36.3 Function Documentation	132
7.36.3.1 system_monitoring_register_callback()	132
7.36.3.2 system_monitoring_start()	133
7.36.3.3 system_monitoring_stop()	133
Index	135

Chapter 1

Sample project

(See the README.md file in the upper level 'examples' directory for more information about examples.)

This is the simplest buildable example. The example is used by command `idf.py create-project` that copies the project to user specified path and set it's name. For more information follow the [docs page](#)

1.1 How to use example

We encourage the users to use the example as a template for the new projects. A recommended way is to follow the instructions on a [docs page](#).

1.2 Example folder contents

The project **sample_project** contains one source file in C language `main.c`. The file is located in folder `[main](main)`.

ESP-IDF projects are built using CMake. The project build configuration is contained in `CMakeLists.txt` files that provide set of directives and instructions describing the project's source files and targets (executable, library, or both).

Below is short explanation of remaining files in the project folder.

```
|— CMakeLists.txt
|— main
-  |— CMakeLists.txt
-  |— main.c
|— README.md
```

This is the file you are currently reading

Additionally, the sample project contains Makefile and component.mk files, used for the legacy Make based build system. They are not used or needed when building with CMake and `idf.py`.

1.3 ESP IDF

Esp idf version `v4.4.1` eclipse [plugin](#)

1.4 Compilation

```
rm -rf build
get_idf
idf.py set-target esp32s3
idf.py all
```

1.5 Flash

```
idf.py flash monitor
```

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

FreeRTOS Defines	9
Project Configuration	11

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

arg_pair	Structure representing an argument pair (type and value)	17
async_resp_arg		18
http_file_server_data		18
i2c_dev_t	Structure representing an I2C device	18
pivot_sectors	Configuration angles	18
rtc_i2c_dev_t		19

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/ actuation_app.c	
Actuation control class responsible for managing actuator actions	21
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/ comm_app.c	
Communication class control application	25
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/ data_app.c	
Memory control application	27
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/ rtc_app.c	
RTC application implementation for managing date and time functionality	47
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/ actuation_app.h	
Actuation control class header file	31
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/ comm_app.h	
Communication class control application	36
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/ data_app.h	
Memory control application	39
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/ rtc_app.h	
RTC application class for managing date and time functionality	44
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gpio_actuator/ gpio_actuator.c	
Implementation of GPIO actuator control functions	51
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gpio_actuator/include/ gpio_actuator.h	
Board GPIO macros and pinout definitions for the GPIO Actuator module	56
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gprs/include/ gprs_uart.h	
Header file for GPRS UART communication functions	64
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/http_server/include/ http_↵ api.h	??
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/ idp_parser.c	
Implementation of the IDP (Intelligent Device Protocol) parser	67
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/include/ idp_parser.h	
IDP Parser class for handling IDP protocol packets	71
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/nvs_data/ nvs_data.c	
Class for direct access to flash memory (NVS)	80
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/nvs_data/include/ nvs_data.h	
Class for direct access to flash memory (NVS)	77
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rf_module/ rf_uart.c	
RF UART module implementation	85
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rf_module/include/ rf_uart.h	
RF UART module functions and callbacks	82

/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/ rtc_ds3231.c	
RTC DS3231 class implementation	95
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/ rtc_i2c_dev.c	
RTC I2C device implementation	100
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/include/ rtc_ds3231.h	
RTC DS3231 class for interfacing with the DS3231 RTC module	87
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/include/ rtc_i2c_dev.h	??
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/ FreeRTOS_defines.h	
Defines for FreeRTOS tasks and priorities	103
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/ log.h	
Creating customizable logs	105
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/ project_config.h	
Configuration and parameter definitions for the project	108
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/wifi_app/ wifi_app.c	
Wi-Fi application implementation	112
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/wifi_app/include/ wifi_app.h	
Wi-Fi application functions and callbacks	110
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/ eco_mode.c	
Implementation of the Eco Mode module	114
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/ main.c	
Main source file for the application	125
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/ scheduling.c	
Implementation of the Scheduling module	126
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/ sectorization.c	
Implementation file for sectorization functionality	128
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/ system_manager.c	
Source file for the system manager module	129
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/ system_monitoring.c	
System monitoring functionality implementation	131
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/ eco_mode.h	
Header file for the Eco Mode module	116
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/ scheduling.h	
Header file for scheduling functionality	118
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/ sectorization.h	
Header file for sectorization functionality	120
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/ system_manager.h	
Header file for system manager module	122
/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/ system_monitoring.h	
Header file for the system monitoring module	123

Chapter 5

Module Documentation

5.1 FreeRTOS Defines

Macros

- `#define SECTORIZATION_TASK_NAME "sectorization task"`
Name of the main sectorization task.
- `#define SECTORIZATION_TASK_SIZE (configMINIMAL_STACK_SIZE * 4)`
Stack size of the main sectorization task.
- `#define SECTORIZATION_TASK_PRIORITY (tskIDLE_PRIORITY + 4)`
Priority of the main sectorization task.
- `#define MAIN_APP_TASK_2_NAME "peak hours task"`
Name of the main peak hours task.
- `#define MAIN_APP_STACK_2_SIZE (configMINIMAL_STACK_SIZE * 4)`
Stack size of the main peak hours task.
- `#define MAIN_APP_TASK_2_PRIORITY (tskIDLE_PRIORITY + 3)`
Priority of the main peak hours task.
- `#define SCHEDULING_TASK_NAME "scheduling task"`
Name of the main scheduling task.
- `#define SCHEDULING_TASK_SIZE (configMINIMAL_STACK_SIZE * 6)`
Stack size of the main scheduling task.
- `#define SCHEDULING_TASK_PRIORITY (tskIDLE_PRIORITY + 3)`
Priority of the main scheduling task.
- `#define DATA_APP_TASK_NAME "data app task"`
Name of the data app task.
- `#define DATA_APP_STACK_SIZE (configMINIMAL_STACK_SIZE * 6)`
Stack size of the data app task.
- `#define DATA_APP_TASK_PRIORITY (tskIDLE_PRIORITY + 2)`
Priority of the data app task.
- `#define COMM_APP_TASK_NAME "comm app task"`
Name of the comm app task.
- `#define COMM_APP_STACK_SIZE (configMINIMAL_STACK_SIZE * 10)`
Stack size of the comm app task.
- `#define COMM_APP_TASK_PRIORITY (tskIDLE_PRIORITY + 2)`
Priority of the comm app task.
- `#define ACTUATION_APP_TASK_NAME "actuation app task"`

- Name of the actuation app task.*

 - #define `ACTUATION_APP_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)

Stack size of the actuation app task.
- #define `ACTUATION_APP_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)

Priority of the actuation app task.
- #define `ACTUATOR_CHECK_TASK_NAME` "actuator check task"

Name of the actuator check task.
- #define `ACTUATOR_CHECK_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)

Stack size of the actuator check task.
- #define `ACTUATOR_CHECK_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)

Priority of the actuator check task.
- #define `ACTUATOR_PERCENT_TASK_NAME` "actuator percent task"

Name of the actuator percent task.
- #define `ACTUATOR_PERCENT_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)

Stack size of the actuator percent task.
- #define `ACTUATOR_PERCENT_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)

Priority of the actuator percent task.
- #define `GPRS_UART_TASK_NAME` "GPRS UART task"

Name of the GPRS UART task.
- #define `GPRS_UART_STACK_SIZE` (configMINIMAL_STACK_SIZE * 10)

Stack size of the GPRS UART task.
- #define `GPRS_UART_TASK_PRIORITY` (tskIDLE_PRIORITY + 12)

Priority of the GPRS UART task.
- #define `RF_UART_TASK_NAME` "RF UART task"

Name of the RF UART task.
- #define `RF_UART_STACK_SIZE` (configMINIMAL_STACK_SIZE * 10)

Stack size of the RF UART task.
- #define `RF_UART_TASK_PRIORITY` (tskIDLE_PRIORITY + 12)

Priority of the RF UART task.
- #define `WIFI_APP_TASK_NAME` "wifi app task"

Name of the Wi-Fi app task.
- #define `WIFI_APP_STACK_SIZE` (configMINIMAL_STACK_SIZE * 4)

Stack size of the Wi-Fi app task.
- #define `WIFI_APP_TASK_PRIORITY` (tskIDLE_PRIORITY + 8)

Priority of the Wi-Fi app task.
- #define `SYSTEM_MONITORING_TASK_NAME` "monitoring task"

Name of the system monitoring task.
- #define `SYSTEM_MONITORING_TASK_SIZE` (configMINIMAL_STACK_SIZE * 6)

Stack size of the system monitoring task.
- #define `SYSTEM_MONITORING_TASK_PRIORITY` (tskIDLE_PRIORITY + 2)

Priority of the system monitoring task.
- #define `ECO_MODE_TASK_NAME` "eco mode task"

Name of the Eco Mode task.
- #define `ECO_MODE_TASK_SIZE` (configMINIMAL_STACK_SIZE * 4)

Stack size of the Eco Mode task.
- #define `ECO_MODE_TASK_PRIORITY` (tskIDLE_PRIORITY + 2)

Priority of the Eco Mode task.

5.1.1 Detailed Description

5.2 Project Configuration

Data Structures

- struct `pivot_sectors`
Configuration angles.

Macros

- #define `CONFIG_FW_VERSION` ("v2.0.0")
Firmware version.
- #define `CONFIG_SCHEDULING_MAX_VALUE` (10)
Maximum number of scheduling values.
- #define `CONFIG_HISTORY_MAX_VALUE` (20)
Maximum number of history values.
- #define `CONFIG_SECTORS_MAX_VALUE` (04)
Maximum number of sectors.
- #define `CONFIG_ACTIONS_UNDEF_VALUE` (655)
Undefined value for actions.
- #define `CONFIG_HTTP_ERROR` ("400")
HTTP error response code.
- #define `CONFIG_HTTP_OK` ("200")
HTTP OK response code.

Typedefs

- typedef void(* `app_callback`) (const char *buffer_request, `comm_type` communication)
Application callback function.

Enumerations

- enum `idp_type` {
 `IDP_0` = 0 , `IDP_1` , `IDP_2` , `IDP_3` ,
 `IDP_4` , `IDP_5` , `IDP_6` , `IDP_7` ,
 `IDP_8` , `IDP_9` , `IDP_10` , `IDP_11` ,
 `IDP_12` , `IDP_13` , `IDP_14` , `IDP_15` ,
 `IDP_16` , `IDP_17` , `IDP_18` , `IDP_22` = 22 ,
 `IDP_30` = 30 , `IDP_90` = 90 , `IDP_91` , `IDP_92` ,
 `IDP_INVALID` = 255 }
Enumerates different packet identifiers for communication.
- enum `comm_type` { `COMM_HTTP_POST` = 0 , `COMM_HTTP_GET` , `COMM_MQTT` }
Enumerates different communication types.
- enum `pivot_states` {
 `PIVOT_ON` = 1 , `PIVOT_OFF` = 2 , `PIVOT_CW` = 3 , `PIVOT_CCW` = 4 ,
 `PIVOT_DRY` = 5 , `PIVOT_WET` = 6 , `PIVOT_PRESSURIZING` = 7 , `PIVOT_UNKNOWN` = 8 }
Pivot actions states.

Functions

- struct `__attribute__((__packed__))`
Pivot actions parameters.

Variables

- `pivot_actions`
- `pivot_config`
- `network_config`
- `eco_mode_config`
- `sector_config`
- `pivot_return_config`
- `pivot_scheduling_date`
- `pivot_scheduling_off_date`
- `pivot_scheduling_angle`
- `pivot_scheduling_off_angle`
- `pivot_history`
- `uint16_t global_angle`
Global angle variable.

5.2.1 Detailed Description

5.2.2 Macro Definition Documentation

5.2.2.1 CONFIG_HISTORY_MAX_VALUE

```
#define CONFIG_HISTORY_MAX_VALUE (20)
```

Maximum number of history values.

This define specifies the maximum number of history values allowed in the project. It can be used to define the size of arrays or data structures related to history.

5.2.2.2 CONFIG_SCHEDULING_MAX_VALUE

```
#define CONFIG_SCHEDULING_MAX_VALUE (10)
```

Maximum number of scheduling values.

This define specifies the maximum number of scheduling values allowed in the project. It can be used to define the size of arrays or data structures related to scheduling.

5.2.2.3 CONFIG_SECTORS_MAX_VALUE

```
#define CONFIG_SECTORS_MAX_VALUE (04)
```

Maximum number of sectors.

This define specifies the maximum number of sectors allowed in the project. It can be used to define the size of arrays or data structures related to sectors.

5.2.3 Typedef Documentation

5.2.3.1 app_callback

```
typedef void(* app_callback) (const char *buffer_request, comm_type communication)
```

Application callback function.

Function signature for the application callback function.

Parameters

<i>buffer_request</i>	The buffer containing the data associated with the request.
<i>communication</i>	The type of communication (HTTP POST, HTTP GET, MQTT).

5.2.4 Enumeration Type Documentation

5.2.4.1 comm_type

```
enum comm_type
```

Enumerates different communication types.

This enum defines communication types used for packet transmission.

5.2.4.2 idp_type

```
enum idp_type
```

Enumerates different packet identifiers for communication.

This enum defines packet identifiers used for communication. Each identifier represents a specific action or configuration parameter.

5.2.4.3 pivot_states

enum `pivot_states`

Pivot actions states.

Enumeration defining pivot actions states.

Enumerator

PIVOT_ON	Pivot on
PIVOT_OFF	Pivot off
PIVOT_CW	Pivot in ClockWise mode (advanced)
PIVOT_CCW	Pivot in Counter ClockWise mode (reverse)
PIVOT_DRY	Irrigation off
PIVOT_WET	Irrigation on
PIVOT_PRESSURIZING	Pivot pressurizing
PIVOT_UNKNOWN	Unknown pivot state

5.2.5 Function Documentation

5.2.5.1 __attribute__()

```
struct __attribute__ (
    (__packed__) )
```

Pivot actions parameters.

History parameters.

Scheduling angle parameters.

Scheduling date parameters.

Configuration parameters.

Structure defining pivot actions parameters.

Structure defining the configuration parameters.

Structure defining the network configuration parameters.

Structure defining the eco mode configuration parameters.

Structure defining the sectors configuration parameters.

Structure defining the scheduling parameters based on date.

Structure defining the scheduling parameters based on angle.

Structure defining the history parameters. < Power state of the pivot (PIVOT_ON or PIVOT_OFF)

< Rotation mode of the pivot (PIVOT_CW or PIVOT_CCW)

< Watering state of the pivot (PIVOT_DRY or PIVOT_WET)

< Percentage value from 0 to 100

< Contactor type

< Pressure switch type

< Pressurization time

< On/off time

< Read time

< GPRS ID

< Start time

< End time

< Array of pivot sectors

< Start angle of the configuration

< End angle of the configuration

< Scheduling ID

< Start date

< End date

< Pivot actions

< Scheduling ID

< End date

< Scheduling ID

< Start date

< End angle

< Pivot actions

< Scheduling ID

< End angle

< Pivot actions

< Start angle

< End angle

< Start date

< End date

5.2.6 Variable Documentation

5.2.6.1 global_angle

`global_angle` [extern]

Global angle variable.

Global variable for the current angle.

Chapter 6

Data Structure Documentation

6.1 `arg_pair` Struct Reference

Structure representing an argument pair (type and value).

```
#include <idp_parser.h>
```

Data Fields

- `const char *` [type](#)
- `void *` [value](#)

6.1.1 Detailed Description

Structure representing an argument pair (type and value).

6.1.2 Field Documentation

6.1.2.1 `type`

```
const char* type
```

Type of the argument.

6.1.2.2 `value`

```
void* value
```

Value of the argument.

The documentation for this struct was generated from the following file:

- /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/include/idp_parser.h

6.2 `async_resp_arg` Struct Reference

Data Fields

- `httpd_handle_t` **hd**
- `int` **fd**

The documentation for this struct was generated from the following file:

- `/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/http_server/http_api.c`

6.3 `http_file_server_data` Struct Reference

Data Fields

- `char` **base_path** [ESP_VFS_PATH_MAX+1]
- `char` **scratch** [HTTP_SCRATCH_BUFSIZE]

6.3.1 Detailed Description

Data structure for HTTP transmission

The documentation for this struct was generated from the following file:

- `/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/http_server/http_api.c`

6.4 `i2c_dev_t` Struct Reference

Structure representing an I2C device.

```
#include <rtc_i2c_dev.h>
```

6.4.1 Detailed Description

Structure representing an I2C device.

The documentation for this struct was generated from the following file:

- `/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/include/rtc_i2c_dev.h`

6.5 `pivot_sectors` Struct Reference

Configuration angles.

```
#include <project_config.h>
```

Data Fields

- uint16_t [start_angle](#)
- uint16_t [end_angle](#)

6.5.1 Detailed Description

Configuration angles.

Structure defining the configuration angles.

6.5.2 Field Documentation

6.5.2.1 end_angle

```
uint16_t end_angle
```

End angle of the configuration

6.5.2.2 start_angle

```
uint16_t start_angle
```

Start angle of the configuration

The documentation for this struct was generated from the following file:

- [/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utis/include/project_config.h](#)

6.6 rtc_i2c_dev_t Struct Reference

Data Fields

- i2c_port_t **port**
- uint8_t **addr**
- gpio_num_t **sda_io_num**
- gpio_num_t **scl_io_num**
- uint32_t **clk_speed**

The documentation for this struct was generated from the following file:

- [/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/include/rtc_i2c_dev.h](#)

Chapter 7

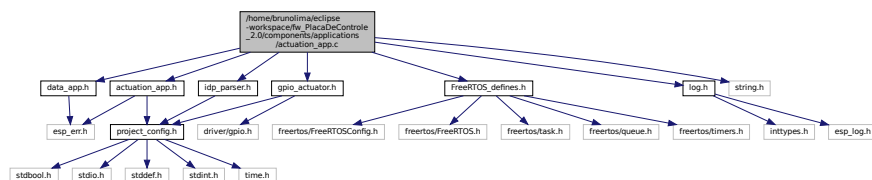
File Documentation

7.1 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/actuation_app.c File Reference

Actuation control class responsible for managing actuator actions.

```
#include "actuation_app.h"
#include "gpio_actuator.h"
#include "data_app.h"
#include "FreeRTOS_defines.h"
#include "idp_parser.h"
#include "log.h"
#include <string.h>
```

Include dependency graph for actuation_app.c:



Macros

- `#define ACTUATION_APP_TAG "actuation_app"`
Tag used for logging within the actuation_app module.
- `#define ACTUATION_APP_POWER_TIME 10000`
Timeout duration for manual power configuration in milliseconds (10 seconds).
- `#define ACTUATION_APP_WATERING_TIME 30000`
Timeout duration for manual watering configuration in milliseconds (30 seconds).
- `#define ACTUATION_APP_ROTATION_TIME 6000`
Timeout duration for manual rotation configuration in milliseconds (6 seconds).
- `#define ACTUATION_APP_PERCENTIMETER_TIME 120000`
Timeout duration for manual percentimeter configuration in milliseconds (2 minutes).

Functions

- void `actuation_app_task` (void *arg)
Task responsible for monitoring possible changes in equipment status.
- void `actuation_app_manual_call` (bool on_off, pivot_actions current_action)
Perform a manual call based on the given parameters.
- esp_err_t `actuation_app_init` (const app_callback callback)
Initialize the actuation control class.
- esp_err_t `actuation_app_set_config` (pivot_config config)
Set the configuration for the actuator.
- void `actuation_app_set_actions` (const pivot_actions config_in, bool alert_change)
Set the actions for the actuator.
- void `actuation_app_get_actions` (pivot_actions *config_out, size_t config_size)
Get the current actions of the actuator.
- void `actuation_app_set_pump` (bool pump_state)
Set the state of the pump.
- void `actuation_app_shutdown` (void)
Shutdown the actuation control class.

Variables

- const pivot_actions `pivot_actions_off`
Pivot actions structure.

7.1.1 Detailed Description

Actuation control class responsible for managing actuator actions.

7.1.2 Function Documentation

7.1.2.1 `actuation_app_get_actions()`

```
void actuation_app_get_actions (
    pivot_actions * config_out,
    size_t config_size )
```

Get the current actions of the actuator.

Reads the current status on IO ports.

Parameters

<code>config_out</code>	[out]: The buffer to store the current pivot actions.
<code>config_size</code>	[in]: The size of the config_out buffer.

7.1.2.2 actuation_app_init()

```
esp_err_t actuation_app_init (  
    const app_callback callback )
```

Initialize the actuation control class.

Initializes the actuation application.

Parameters

<i>callback</i>	[in]: Callback function for actuation events.
-----------------	---

Returns

esp_err_t: Error code indicating the success of the initialization.

7.1.2.3 actuation_app_manual_call()

```
void actuation_app_manual_call (  
    bool on_off,  
    pivot_actions current_action )
```

Perform a manual call based on the given parameters.

Parameters

<i>on_off</i>	[in]: The state to set (true for on, false for off).
<i>current_action</i>	[in]: The current pivot actions.

7.1.2.4 actuation_app_set_actions()

```
void actuation_app_set_actions (  
    const pivot_actions config_in,  
    bool alert_change )
```

Set the actions for the actuator.

Applies the provided actions to the actuation application.

Parameters

<i>config_in</i>	[in]: The pivot actions to set.
<i>alert_change</i>	[in]: Flag indicating whether to alert about manual configuration.

7.1.2.5 actuation_app_set_config()

```
esp_err_t actuation_app_set_config (  
    pivot_config config )
```

Set the configuration for the actuator.

Sets the configuration for the actuation application.

Parameters

<i>config</i>	[in]: The pivot configuration to set.
---------------	---------------------------------------

Returns

esp_err_t: Error code indicating the success of the configuration.

7.1.2.6 actuation_app_set_pump()

```
void actuation_app_set_pump (  
    bool pump_state )
```

Set the state of the pump.

Sets the pump state for the actuation application.

Parameters

<i>pump_state</i>	[in]: The state to set (true for on, false for off).
-------------------	--

7.1.2.7 actuation_app_shutdown()

```
void actuation_app_shutdown (  
    void )
```

Shutdown the actuation control class.

Shuts down the actuation application.

7.1.2.8 actuation_app_task()

```
void actuation_app_task (  
    void * arg )
```

Task responsible for monitoring possible changes in equipment status.

Parameters

<i>arg</i>	[in]: Task argument (default NULL).
<i>arg</i>	- [in]: task argument (default NULL)

7.1.3 Variable Documentation

7.1.3.1 pivot_actions_off

```
const pivot_actions pivot_actions_off
```

Initial value:

```
= {  
    .power_state = PIVOT_OFF,  
    .rotation = PIVOT_CW,  
    .watering_state = PIVOT_DRY,  
    .percentimeter = 0  
}
```

Pivot actions structure.

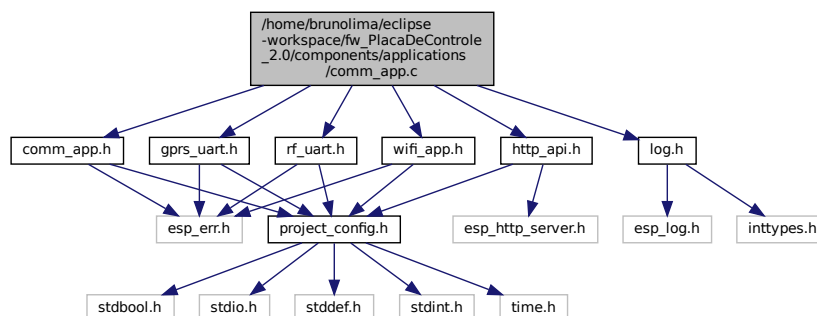
This structure defines the possible actions for actuation, including power state, rotation direction, watering state, and percentimeter reading.

7.2 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/comm_app.c File Reference

Communication class control application.

```
#include "comm_app.h"  
#include "gprs_uart.h"  
#include "rf_uart.h"  
#include "wifi_app.h"  
#include "http_api.h"  
#include "log.h"
```

Include dependency graph for comm_app.c:



Macros

- `#define COMM_APP_TAG "comm_app"`
Tag used for logging within the comm_app module.

Functions

- `esp_err_t comm_app_init (const app_callback callback)`
Initializes the communication class.
- `void comm_app_send_idp_pack (const char *idp_pack, comm_type communication)`
Sends an IDP package using the specified communication type.
- `void comm_app_wifi_config (char *wifi_ssid, char *wifi_pass)`
Configures the Wi-Fi settings.

7.2.1 Detailed Description

Communication class control application.

Date

June 23, 2022

7.2.2 Function Documentation

7.2.2.1 comm_app_init()

```
esp_err_t comm_app_init (
    const app_callback callback )
```

Initializes the communication class.

Initializes all communication modules.

Parameters

<i>callback</i>	[in]: Callback function for communication events.
-----------------	---

Returns

`esp_err_t`: Error code indicating the success of the initialization.

This function initializes various communication modules and registers a callback function to be used for communication events.

7.2.2.2 comm_app_send_idp_pack()

```
void comm_app_send_idp_pack (
    const char * idp_pack,
    comm_type communication )
```

Sends an IDP package using the specified communication type.

Parameters

<i>idp_pack</i>	[in]: IDP package to send.
<i>communication</i>	[in]: Type of communication (e.g., COMM_MQTT, COMM_HTTP_POST).

This function sends an IDP package using the specified communication type. If the communication type is HTTP, it uses the HTTP server; if MQTT, it uses GPRS and RF UART.

7.2.2.3 comm_app_wifi_config()

```
void comm_app_wifi_config (
    char * wifi_ssid,
    char * wifi_pass )
```

Configures the Wi-Fi settings.

Parameters

<i>wifi_ssid</i>	[in]: Wi-Fi SSID.
<i>wifi_pass</i>	[in]: Wi-Fi password.

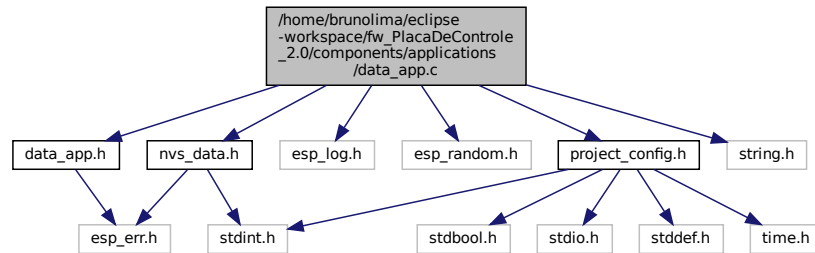
This function configures the Wi-Fi settings for the communication module.

7.3 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/data_app.c File Reference

Memory control application.

```
#include "data_app.h"
#include "nvs_data.h"
#include "esp_log.h"
#include "esp_random.h"
#include "project_config.h"
#include <string.h>
```

Include dependency graph for data_app.c:



Macros

- `#define DATA_APP_TAG "data_app"`
Class log tag.
- `#define DATA_ACTION "action"`
NVS access space for action data.
- `#define DATA_PIVOT_CONFIG "pivot_config"`
NVS access space for pivot configuration data.
- `#define DATA_NETWORK_CONFIG "net_config"`
NVS access space for network configuration data.
- `#define DATA_ECO_MODE_CONFIG "eco_config"`
NVS access space for eco mode configuration data.
- `#define DATA_SECTOR_CONFIG "sector_config"`
NVS access space for sector configuration data.
- `#define DATA_RETURN_CONFIG "return_config"`
NVS access space for return configuration data.
- `#define DATA_SCHEDULING_DATE "s_date"`
NVS access space for scheduling date data.
- `#define DATA_SCHEDULING_OFF_DATE "s_off_date"`
NVS access space for scheduling off date data.
- `#define DATA_SCHEDULING_ANGLE "s_angle"`
NVS access space for scheduling angle data.
- `#define DATA_SCHEDULING_OFF_ANGLE "s_off_angle"`
NVS access space for scheduling off angle data.
- `#define DATA_HISTORY "history"`
NVS access space for history data.
- `#define DATA_TIMESTAMP "timestamp"`
NVS access space for timestamp data.

Functions

- `esp_err_t data_app_init (void)`
Initializes the data application.
- `esp_err_t data_app_save (data_type_t data_type, const void *data, size_t data_size)`
Saves data to the NVS based on the specified data type.

- `esp_err_t data_app_load (data_type_t data_type, void *data)`
Loads data from the NVS based on the specified data type.
- `esp_err_t data_app_delete_scheduling (char *scheduling_id)`
Deletes scheduling data based on the provided scheduling ID.
- `void data_app_gen_scheduling_key (char *scheduling_id)`
Generates a scheduling key.
- `size_t data_app_get_data_size (const char *label_name)`
Gets the size of the data stored in the NVS based on the label name.

7.3.1 Detailed Description

Memory control application.

Date

Set 18, 2022

7.3.2 Function Documentation

7.3.2.1 data_app_delete_scheduling()

```
esp_err_t data_app_delete_scheduling (  
    char * scheduling_id )
```

Deletes scheduling data based on the provided scheduling ID.

Deletes scheduling data based on the scheduling ID.

Parameters

<i>scheduling_id</i>	Scheduling ID to identify the data to be deleted.
----------------------	---

Returns

`esp_err_t` Error code indicating the success of the operation.

7.3.2.2 data_app_gen_scheduling_key()

```
void data_app_gen_scheduling_key (  
    char * scheduling_id )
```

Generates a scheduling key.

Generates a scheduling key based on the current timestamp.

Parameters

<i>scheduling</i> <i>_id</i>	Pointer to store the generated scheduling key.
---------------------------------	--

7.3.2.3 data_app_get_data_size()

```
size_t data_app_get_data_size (
    const char * label_name )
```

Gets the size of the data stored in the NVS based on the label name.

Gets the size of data associated with a given label name.

Parameters

<i>label_name</i>	Label name to identify the data.
-------------------	----------------------------------

Returns

size_t Size of the data.

7.3.2.4 data_app_init()

```
esp_err_t data_app_init (
    void )
```

Initializes the data application.

Returns

esp_err_t Error code indicating the success of the operation.

7.3.2.5 data_app_load()

```
esp_err_t data_app_load (
    data_type_t data_type,
    void * data )
```

Loads data from the NVS based on the specified data type.

Loads data of a specific type.

Parameters

<i>data_type</i>	Data type identifier.
<i>data</i>	Pointer to store the loaded data.

Returns

esp_err_t Error code indicating the success of the operation.

7.3.2.6 data_app_save()

```
esp_err_t data_app_save (
    data_type_t data_type,
    const void * data,
    size_t data_size )
```

Saves data to the NVS based on the specified data type.

Saves data of a specific type.

Parameters

<i>data_type</i>	Data type identifier.
<i>data</i>	Pointer to the data to be saved.
<i>data_size</i>	Size of the data.

Returns

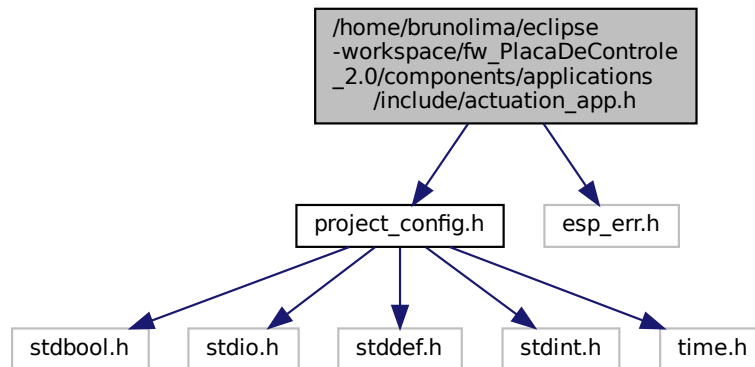
esp_err_t Error code indicating the success of the operation.

7.4 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_↵ 2.0/components/applications/include/actuation_app.h File Reference

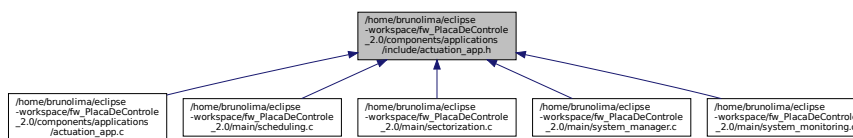
Actuation control class header file.

```
#include "project_config.h"
#include "esp_err.h"
```

Include dependency graph for `actuation_app.h`:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t actuation_app_init` (const `app_callback` callback)
Initializes the actuation application.
- `esp_err_t actuation_app_set_config` (pivot_config config)
Sets the configuration for the actuation application.
- `void actuation_app_set_actions` (const pivot_actions config_in, bool alert_change)
Applies the provided actions to the actuation application.
- `void actuation_app_get_actions` (pivot_actions *config_out, size_t config_size)
Reads the current status on IO ports.
- `void actuation_app_set_pump` (bool pump_state)
Sets the pump state for the actuation application.
- `void actuation_app_shutdown` (void)
Shuts down the actuation application.

Variables

- const pivot_actions `pivot_actions_off`
Pivot actions structure.

7.4.1 Detailed Description

Actuation control class header file.

Date

June 23, 2022

7.4.2 Function Documentation

7.4.2.1 actuation_app_get_actions()

```
void actuation_app_get_actions (
    pivot_actions * config_out,
    size_t config_size )
```

Reads the current status on IO ports.

This function retrieves the current status of the actuation application, including power state, rotation direction, watering state, and percentimeter reading.

Parameters

<i>config_out</i>	- [out]: Pivot configuration structure.
<i>config_size</i>	- [out]: Structure size.

Reads the current status on IO ports.

Parameters

<i>config_out</i>	[out]: The buffer to store the current pivot actions.
<i>config_size</i>	[in]: The size of the config_out buffer.

7.4.2.2 actuation_app_init()

```
esp_err_t actuation_app_init (
    const app_callback callback )
```

Initializes the actuation application.

This function starts the tasks for reading and actuation of the I/O ports.

Parameters

<i>app_callback</i>	- [in]: Function used for returning to the main application class.
---------------------	--

Returns

- ESP_OK: Success
- ESP_FAIL: Fail to initialize

Initializes the actuation application.

Parameters

<i>callback</i>	[in]: Callback function for actuation events.
-----------------	---

Returns

esp_err_t: Error code indicating the success of the initialization.

7.4.2.3 actuation_app_set_actions()

```
void actuation_app_set_actions (
    const pivot_actions config_in,
    bool alert_change )
```

Applies the provided actions to the actuation application.

This function applies the provided actions to the actuation application, controlling power state, rotation direction, watering state, and percentimeter reading.

Parameters

<i>config_in</i>	- [in]: Pivot actions structure.
<i>alert_change</i>	- [in]: Flag indicating whether to alert on configuration change.

Applies the provided actions to the actuation application.

Parameters

<i>config_in</i>	[in]: The pivot actions to set.
<i>alert_change</i>	[in]: Flag indicating whether to alert about manual configuration.

7.4.2.4 actuation_app_set_config()

```
esp_err_t actuation_app_set_config (
    pivot_config config )
```

Sets the configuration for the actuation application.

This function applies the provided configuration to the drives.

Parameters

<i>config</i>	- [in]: Pivot configuration structure.
---------------	--

Returns

- ESP_OK: Configuration applied successfully
- ESP_FAIL: Configuration failed

Sets the configuration for the actuation application.

Parameters

<i>config</i>	[in]: The pivot configuration to set.
---------------	---------------------------------------

Returns

esp_err_t: Error code indicating the success of the configuration.

7.4.2.5 actuation_app_set_pump()

```
void actuation_app_set_pump (  
    bool pump_state )
```

Sets the pump state for the actuation application.

This function turns on or off the pump of the actuation application.

Parameters

<i>pump_state</i>	- [in]: Pump state (true for on, false for off).
-------------------	--

Sets the pump state for the actuation application.

Parameters

<i>pump_state</i>	[in]: The state to set (true for on, false for off).
-------------------	--

7.4.2.6 actuation_app_shutdown()

```
void actuation_app_shutdown (  
    void )
```

Shuts down the actuation application.

This function gracefully shuts down the actuation application.

Shuts down the actuation application.

7.4.3 Variable Documentation

7.4.3.1 pivot_actions_off

```
const pivot_actions pivot_actions_off [extern]
```

Pivot actions structure.

This structure defines the possible actions for actuation, including power state, rotation direction, watering state, and percentimeter reading.

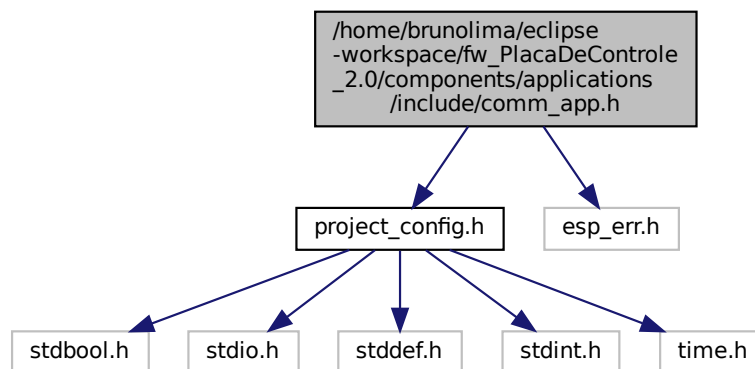
7.5 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/comm_app.h File Reference

Communication class control application.

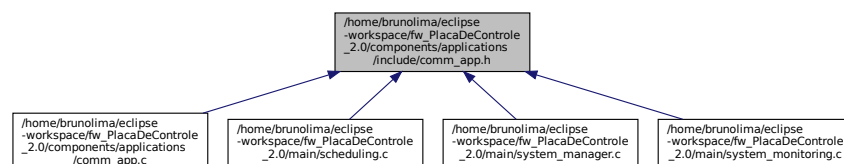
```
#include "project_config.h"
```

```
#include "esp_err.h"
```

Include dependency graph for comm_app.h:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t comm_app_init` (const `app_callback` callback)
Initializes all communication modules.
- void `comm_app_send_idp_pack` (const char *idp_pack, `comm_type` communication)
Sends an IDP package using the specified communication type.
- void `comm_app_wifi_config` (char *wifi_ssid, char *wifi_pass)
Configures the Wi-Fi settings.

7.5.1 Detailed Description

Communication class control application.

Date

June 23, 2022

7.5.2 Function Documentation

7.5.2.1 `comm_app_init()`

```
esp_err_t comm_app_init (  
    const app_callback callback )
```

Initializes all communication modules.

Parameters

<i>callback</i>	[in]: Function used with return to the main application class.
-----------------	--

Returns

- ESP_OK: Success
- ESP_FAIL: Fail

This function initializes all communication modules and registers a callback function to be used in the main application class.

Initializes all communication modules.

Parameters

<i>callback</i>	[in]: Callback function for communication events.
-----------------	---

Returns

`esp_err_t`: Error code indicating the success of the initialization.

This function initializes various communication modules and registers a callback function to be used for communication events.

7.5.2.2 comm_app_send_idp_pack()

```
void comm_app_send_idp_pack (
    const char * idp_pack,
    comm_type communication )
```

Sends an IDP package using the specified communication type.

Parameters

<i>idp_pack</i>	[in]: IDP package to send.
<i>communication</i>	[in]: Type of communication (e.g., COMM_MQTT, COMM_WIFI).

This function sends an IDP package using the specified communication type.

Parameters

<i>idp_pack</i>	[in]: IDP package to send.
<i>communication</i>	[in]: Type of communication (e.g., COMM_MQTT, COMM_HTTP_POST).

This function sends an IDP package using the specified communication type. If the communication type is HTTP, it uses the HTTP server; if MQTT, it uses GPRS and RF UART.

7.5.2.3 comm_app_wifi_config()

```
void comm_app_wifi_config (
    char * wifi_ssid,
    char * wifi_pass )
```

Configures the Wi-Fi settings.

Parameters

<i>wifi_ssid</i>	[in]: Wi-Fi SSID.
<i>wifi_pass</i>	[in]: Wi-Fi password.

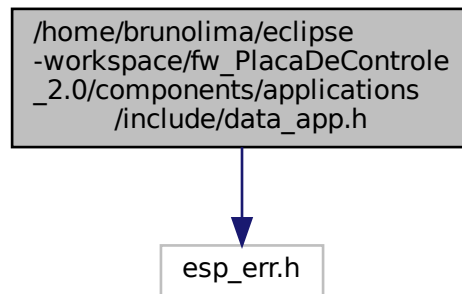
This function configures the Wi-Fi settings for the communication module.

7.6 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/data_app.h File Reference

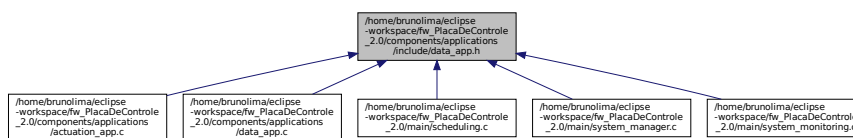
Memory control application.

```
#include "esp_err.h"
```

Include dependency graph for data_app.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `data_type_t` {
`DATA_TYPE_ACTIONS` = 0 , `DATA_TYPE_PIVOT_CONFIG` , `DATA_TYPE_NETWORK_CONFIG` ,
`DATA_TYPE_ECO_MODE_CONFIG` ,
`DATA_TYPE_SECTOR_CONFIG` , `DATA_TYPE_RETURN_CONFIG` , `DATA_TYPE_SCHEDULING_DATE` ,
`DATA_TYPE_SCHEDULING_OFF_DATE` ,
`DATA_TYPE_SCHEDULING_ANGLE` , `DATA_TYPE_SCHEDULING_OFF_ANGLE` , `DATA_TYPE_HISTORY` ,
`DATA_TYPE_OLD_HISTORY` ,
`DATA_TYPE_TIMESTAMP` }

Enumeration defining various data types used in the application.

Functions

- `esp_err_t data_app_init` (void)
Initializes the data application.
- `esp_err_t data_app_save` (`data_type_t` data_type, const void *data, size_t data_size)

Saves data of a specific type.

- `esp_err_t data_app_load (data_type_t data_type, void *data)`

Loads data of a specific type.

- `esp_err_t data_app_delete_scheduling (char *scheduling_id)`

Deletes scheduling data based on the scheduling ID.

- `void data_app_gen_scheduling_key (char *scheduling_id)`

Generates a scheduling key based on the current timestamp.

- `size_t data_app_get_data_size (const char *label_name)`

Gets the size of data associated with a given label name.

7.6.1 Detailed Description

Memory control application.

Date

June 15, 2022

7.6.2 Enumeration Type Documentation

7.6.2.1 data_type_t

`enum data_type_t`

Enumeration defining various data types used in the application.

This enumeration represents the different types of data that can be managed by the data application. Each value corresponds to a specific data type.

Enumerator

DATA_TYPE_ACTIONS	Actions data type.
DATA_TYPE_PIVOT_CONFIG	Pivot configuration data type.
DATA_TYPE_NETWORK_CONFIG	Network configuration data type.
DATA_TYPE_ECO_MODE_CONFIG	Eco mode configuration data type.
DATA_TYPE_SECTOR_CONFIG	Sector configuration data type.
DATA_TYPE_RETURN_CONFIG	Return configuration data type.
DATA_TYPE_SCHEDULING_DATE	Scheduling date data type.
DATA_TYPE_SCHEDULING_OFF_DATE	Scheduling off date data type.
DATA_TYPE_SCHEDULING_ANGLE	Scheduling angle data type.
DATA_TYPE_SCHEDULING_OFF_ANGLE	Scheduling off angle data type.
DATA_TYPE_HISTORY	History data type.
DATA_TYPE_OLD_HISTORY	Old history data type.
DATA_TYPE_TIMESTAMP	Timestamp data type.

7.6.3 Function Documentation

7.6.3.1 data_app_delete_scheduling()

```
esp_err_t data_app_delete_scheduling (
    char * scheduling_id )
```

Deletes scheduling data based on the scheduling ID.

Parameters

<i>scheduling_id</i>	[in]: Scheduling ID to delete.
----------------------	--------------------------------

Returns

esp_err_t: Error code indicating the success of the delete operation.

This function deletes scheduling data based on the scheduling ID.

Deletes scheduling data based on the scheduling ID.

Parameters

<i>scheduling_id</i>	Scheduling ID to identify the data to be deleted.
----------------------	---

Returns

esp_err_t Error code indicating the success of the operation.

7.6.3.2 data_app_gen_scheduling_key()

```
void data_app_gen_scheduling_key (
    char * scheduling_id )
```

Generates a scheduling key based on the current timestamp.

Parameters

<i>scheduling_id</i>	[out]: Buffer to store the generated scheduling ID.
----------------------	---

This function generates a scheduling key based on the current timestamp.

Generates a scheduling key based on the current timestamp.

Parameters

<i>scheduling</i> ↔ _id	Pointer to store the generated scheduling key.
----------------------------	--

7.6.3.3 data_app_get_data_size()

```
size_t data_app_get_data_size (
    const char * label_name )
```

Gets the size of data associated with a given label name.

Parameters

<i>label_name</i>	[in]: Label name for which to get the data size.
-------------------	--

Returns

size_t: Size of the data associated with the label name.

This function gets the size of data associated with a given label name.

Gets the size of data associated with a given label name.

Parameters

<i>label_name</i>	Label name to identify the data.
-------------------	----------------------------------

Returns

size_t Size of the data.

7.6.3.4 data_app_init()

```
esp_err_t data_app_init (
    void )
```

Initializes the data application.

Returns

esp_err_t: Error code indicating the success of the initialization.

This function initializes the data application for managing various types of data.

Returns

`esp_err_t` Error code indicating the success of the operation.

7.6.3.5 data_app_load()

```
esp_err_t data_app_load (  
    data_type_t data_type,  
    void * data )
```

Loads data of a specific type.

Parameters

<i>data_type</i>	[in]: Type of data to load.
<i>data</i>	[out]: Pointer to the buffer for loading data.

Returns

`esp_err_t`: Error code indicating the success of the load operation.

This function loads data of a specific type from the underlying storage.

Loads data of a specific type.

Parameters

<i>data_type</i>	Data type identifier.
<i>data</i>	Pointer to store the loaded data.

Returns

`esp_err_t` Error code indicating the success of the operation.

7.6.3.6 data_app_save()

```
esp_err_t data_app_save (  
    data_type_t data_type,  
    const void * data,  
    size_t data_size )
```

Saves data of a specific type.

Parameters

<i>data_type</i>	[in]: Type of data to save.
<i>data</i>	[in]: Pointer to the data.
<i>data_size</i>	[in]: Size of the data in bytes.

Returns

`esp_err_t`: Error code indicating the success of the save operation.

This function saves data of a specific type to the underlying storage.

Saves data of a specific type.

Parameters

<i>data_type</i>	Data type identifier.
<i>data</i>	Pointer to the data to be saved.
<i>data_size</i>	Size of the data.

Returns

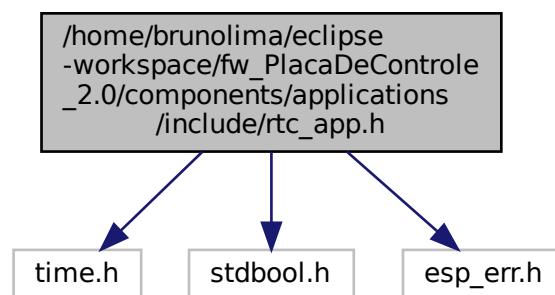
`esp_err_t` Error code indicating the success of the operation.

7.7 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/include/rtc_app.h File Reference

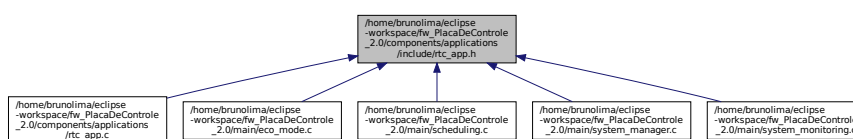
RTC application class for managing date and time functionality.

```
#include <time.h>
#include <stdbool.h>
#include "esp_err.h"
```

Include dependency graph for `rtc_app.h`:



This graph shows which files directly or indirectly include this file:



Macros

- `#define RTC_CONFIG_TIMEZONE -3`
The default time zone for the RTC configuration.

Functions

- `esp_err_t rtc_app_init (void)`
Initializes the RTC application.
- `bool rtc_app_set_timestamp (time_t timestamp)`
Sets the timestamp in the RTC.
- `time_t rtc_app_get_timestamp (bool rtc_show_dt)`
Gets the timestamp from the RTC.
- `void rtc_app_get_date_time (struct tm *rtcinfo)`
Gets the date and time from the RTC.
- `void rtc_show_date_time (time_t timestamp_now, uint8_t time_z)`
Displays the date and time.
- `void rtc_app_get_str_date_time (time_t timestamp_now, char *str_out)`
Gets the date and time in string format.

7.7.1 Detailed Description

RTC application class for managing date and time functionality.

This header file defines the RTC application class for managing date and time functionality.

7.7.2 Function Documentation**7.7.2.1 rtc_app_get_date_time()**

```
void rtc_app_get_date_time (
    struct tm * rtcinfo )
```

Gets the date and time from the RTC.

This function retrieves the date and time from the RTC and stores it in the specified time structure.

Parameters

<i>rtcinfo</i>	The pointer to the time structure to store the date and time.
----------------	---

7.7.2.2 rtc_app_get_str_date_time()

```
void rtc_app_get_str_date_time (
    time_t timestamp_now,
    char * str_out )
```

Gets the date and time in string format.

This function retrieves the date and time from the RTC and formats it as a string.

Parameters

<i>timestamp_now</i>	The timestamp value.
<i>str_out</i>	Pointer to the output string buffer.

7.7.2.3 rtc_app_get_timestamp()

```
time_t rtc_app_get_timestamp (
    bool rtc_show_dt )
```

Gets the timestamp from the RTC.

This function retrieves the timestamp from the RTC.

Parameters

<i>rtc_show_dt</i>	Flag indicating whether to display the date and time in local time.
--------------------	---

Returns

time_t The timestamp value.

7.7.2.4 rtc_app_init()

```
esp_err_t rtc_app_init (
    void )
```

Initializes the RTC application.

This function initializes the RTC application.

Returns

esp_err_t Error code indicating the success of the operation.

This function initializes the RTC application by configuring the RTC I2C device.

Returns

esp_err_t Error code indicating the success of the operation.

7.7.2.5 rtc_app_set_timestamp()

```
bool rtc_app_set_timestamp (
    time_t timestamp )
```

Sets the timestamp in the RTC.

This function sets the timestamp in the RTC with the specified timestamp value.

Parameters

<i>timestamp</i>	The timestamp value to set.
------------------	-----------------------------

Returns

bool True if setting the timestamp is successful, false otherwise.

7.7.2.6 rtc_show_date_time()

```
void rtc_show_date_time (
    time_t timestamp_now,
    uint8_t time_z )
```

Displays the date and time.

This function displays the date and time based on the specified timestamp and time zone.

Parameters

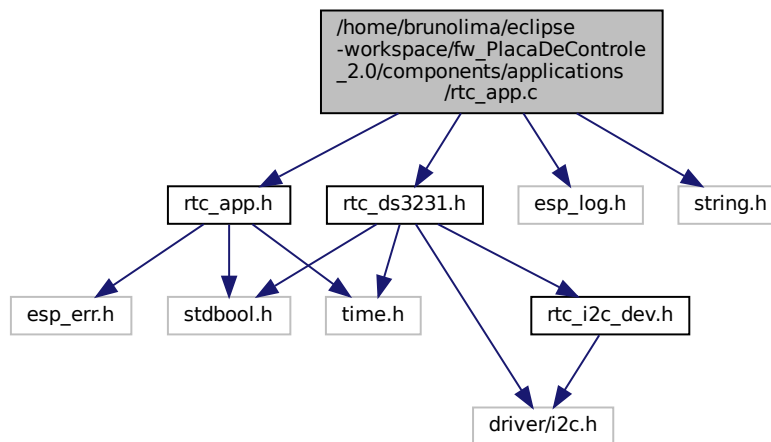
<i>timestamp_now</i>	The timestamp value.
<i>time_z</i>	The time zone.

7.8 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/applications/rtc_app.c File Reference ↩

RTC application implementation for managing date and time functionality.

```
#include "rtc_app.h"
#include "rtc_ds3231.h"
#include "esp_log.h"
#include <string.h>
```

Include dependency graph for rtc_app.c:



Macros

- `#define RTC_APP_TAG "rtc_app"`
Log tag for RTC application.
- `#define RTC_SDA_PIN 36`
The GPIO pin number for the RTC I2C SDA signal.
- `#define RTC_SCL_PIN 37`
The GPIO pin number for the RTC I2C SCL signal.

Functions

- `esp_err_t rtc_app_init (void)`
Initializes the RTC application.
- `bool rtc_app_set_timestamp (time_t timestamp)`
Sets the timestamp in the RTC.
- `time_t rtc_app_get_timestamp (bool rtc_show_dt)`
Gets the timestamp from the RTC.
- `void rtc_app_get_date_time (struct tm *rtcinfo)`
Gets the date and time from the RTC.
- `void rtc_show_date_time (time_t timestamp_now, uint8_t time_z)`
Displays the date and time.
- `void rtc_app_get_str_date_time (time_t timestamp_now, char *str_out)`
Gets the date and time in string format.

7.8.1 Detailed Description

RTC application implementation for managing date and time functionality.

This file contains the implementation of the RTC application for managing date and time functionality.

7.8.2 Function Documentation

7.8.2.1 rtc_app_get_date_time()

```
void rtc_app_get_date_time (
    struct tm * rtcinfo )
```

Gets the date and time from the RTC.

This function retrieves the date and time from the RTC and stores it in the specified time structure.

Parameters

<i>rtcinfo</i>	The pointer to the time structure to store the date and time.
----------------	---

7.8.2.2 rtc_app_get_str_date_time()

```
void rtc_app_get_str_date_time (
    time_t timestamp_now,
    char * str_out )
```

Gets the date and time in string format.

This function retrieves the date and time from the RTC and formats it as a string.

Parameters

<i>timestamp_now</i>	The timestamp value.
<i>str_out</i>	Pointer to the output string buffer.

7.8.2.3 rtc_app_get_timestamp()

```
time_t rtc_app_get_timestamp (
    bool rtc_show_dt )
```

Gets the timestamp from the RTC.

This function retrieves the timestamp from the RTC.

Parameters

<i>rtc_show_dt</i>	Flag indicating whether to display the date and time in local time.
--------------------	---

Returns

`time_t` The timestamp value.

7.8.2.4 rtc_app_init()

```
esp_err_t rtc_app_init (  
    void )
```

Initializes the RTC application.

This function initializes the RTC application by configuring the RTC I2C device.

Returns

`esp_err_t` Error code indicating the success of the operation.

7.8.2.5 rtc_app_set_timestamp()

```
bool rtc_app_set_timestamp (  
    time_t timestamp )
```

Sets the timestamp in the RTC.

This function sets the timestamp in the RTC with the specified timestamp value.

Parameters

<i>timestamp</i>	The timestamp value to set.
------------------	-----------------------------

Returns

`bool` True if setting the timestamp is successful, false otherwise.

7.8.2.6 rtc_show_date_time()

```
void rtc_show_date_time (  
    time_t timestamp_now,  
    uint8_t time_z )
```

Displays the date and time.

This function displays the date and time based on the specified timestamp and time zone.

Parameters

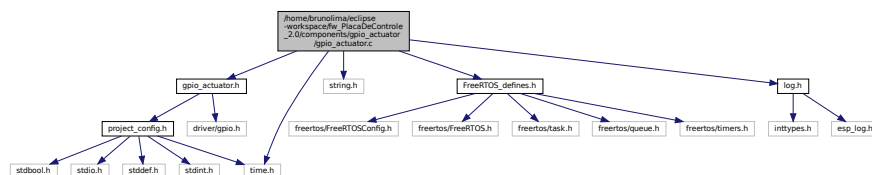
<code>timestamp_now</code>	The timestamp value.
<code>time_z</code>	The time zone.

7.9 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gpio_actuator/gpio_actuator.c File Reference

Implementation of GPIO actuator control functions.

```
#include "gpio_actuator.h"
#include <time.h>
#include <string.h>
#include "FreeRTOS_defines.h"
#include "log.h"
```

Include dependency graph for gpio_actuator.c:



Macros

- `#define GPIO_ACT_TAG "gpio_actuator"`
Tag used for logging in the GPIO actuator module.
- `#define GPIO_ACT_INTR_FLAG_DEFAULT 0`
Default interrupt flag for GPIO handling.

Functions

- void `vPercTimerOnExpire` (TimerHandle_t pxTimer)
Callback function for the expiration of the Perc On timer.
- void `vPercTimerOffExpire` (TimerHandle_t pxTimer)
Callback function for the expiration of the Perc Off timer.
- `esp_err_t gpio_actuator_start` (void)
Start the relays accordingly after pressure check or dry mode.
- void `actuator_wait_pressure` (void *arg)
Water pressure application task.
- void `actuator_read_percent` (void *arg)
Percentimeter reading task.
- `esp_err_t gpio_actuator_init` ()
Initializes the GPIO actuator module.
- `esp_err_t gpio_actuator_config` (pivot_config config)

Configures the GPIO actuator module based on the provided configuration.

- `esp_err_t gpio_actuator_set` (`gpio_actions` actions)

Sets the GPIO actuator actions based on the provided actions.

- `gpio_actions gpio_actuator_get` (void)

Gets the current GPIO actuator state.

- `void gpio_actuator_shutdown` (void)

Shuts down the GPIO actuator module.

- `void gpio_actuator_pump_on` (void)

Turns on the pump in the GPIO actuator module.

- `void gpio_actuator_pump_off` (void)

Turns off the pump in the GPIO actuator module.

- `void gpio_actuator_pressure_on` (void)

Turns on pressure application in the GPIO actuator module.

- `void gpio_actuator_pressure_off` (void)

Turns off pressure application in the GPIO actuator module.

7.9.1 Detailed Description

Implementation of GPIO actuator control functions.

Date

June 20, 2022

7.9.2 Function Documentation

7.9.2.1 actuator_read_percent()

```
void actuator_read_percent (
    void * arg )
```

Percentimeter reading task.

Parameters

<i>arg</i>	Task argument (default NULL)
------------	------------------------------

7.9.2.2 actuator_wait_pressure()

```
void actuator_wait_pressure (
    void * arg )
```

Water pressure application task.

Parameters

<i>arg</i>	Task argument (default NULL)
------------	------------------------------

7.9.2.3 gpio_actuator_config()

```
esp_err_t gpio_actuator_config (  
    pivot_config config )
```

Configures the GPIO actuator module based on the provided configuration.

Configures the GPIO actuator module with the specified configuration.

This function configures the GPIO actuator module based on the provided configuration.

Parameters

<i>config</i>	Pivot configuration.
---------------	----------------------

Returns

esp_err_t Error code indicating the success of the operation.

7.9.2.4 gpio_actuator_get()

```
pivot_actions gpio_actuator_get (  
    void )
```

Gets the current GPIO actuator state.

Gets the current pivot configuration.

This function retrieves the current GPIO actuator state.

Returns

pivot_actions Current pivot actions.

7.9.2.5 gpio_actuator_init()

```
esp_err_t gpio_actuator_init (  
    void )
```

Initializes the GPIO actuator module.

This function initializes the GPIO actuator module, configures GPIO pins, and sets up timers and tasks.

Returns

`esp_err_t` Error code indicating the success of the operation.

7.9.2.6 gpio_actuator_pressure_off()

```
void gpio_actuator_pressure_off (  
    void )
```

Turns off pressure application in the GPIO actuator module.

Turns off the pressure relay and stops pressure monitoring.

7.9.2.7 gpio_actuator_pressure_on()

```
void gpio_actuator_pressure_on (  
    void )
```

Turns on pressure application in the GPIO actuator module.

Turns on the pressure relay and starts pressure monitoring.

7.9.2.8 gpio_actuator_pump_off()

```
void gpio_actuator_pump_off (  
    void )
```

Turns off the pump in the GPIO actuator module.

Turns off the pump relay.

7.9.2.9 gpio_actuator_pump_on()

```
void gpio_actuator_pump_on (  
    void )
```

Turns on the pump in the GPIO actuator module.

Turns on the pump relay.

7.9.2.10 gpio_actuator_set()

```
esp_err_t gpio_actuator_set (  
    pivot_actions actions )
```

Sets the GPIO actuator actions based on the provided actions.

Sets the pivot configuration.

This function sets the GPIO actuator actions based on the provided actions.

Parameters

<i>actions</i>	Pivot actions to set.
----------------	-----------------------

Returns

`esp_err_t` Error code indicating the success of the operation.

7.9.2.11 gpio_actuator_shutdown()

```
void gpio_actuator_shutdown (  
    void )
```

Shuts down the GPIO actuator module.

Shuts down all relays.

This function shuts down the GPIO actuator module by turning off relays and cleaning up resources.

7.9.2.12 gpio_actuator_start()

```
esp_err_t gpio_actuator_start (  
    void )
```

Start the relays accordingly after pressure check or dry mode.

Returns

- ESP_OK: Success
- ESP_FAIL: Fail to initialize

7.9.2.13 vPercTimerOffExpire()

```
void vPercTimerOffExpire (  
    TimerHandle_t pxTimer )
```

Callback function for the expiration of the Perc Off timer.

Parameters

<i>pxTimer</i>	The timer handle
----------------	------------------

7.9.2.14 vPercTimerOnExpire()

```
void vPercTimerOnExpire (
    TimerHandle_t pxTimer )
```

Callback function for the expiration of the Perc On timer.

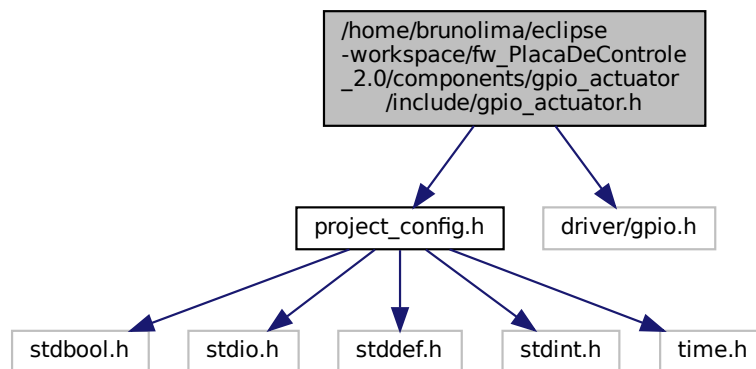
Parameters

<i>pxTimer</i>	The timer handle
----------------	------------------

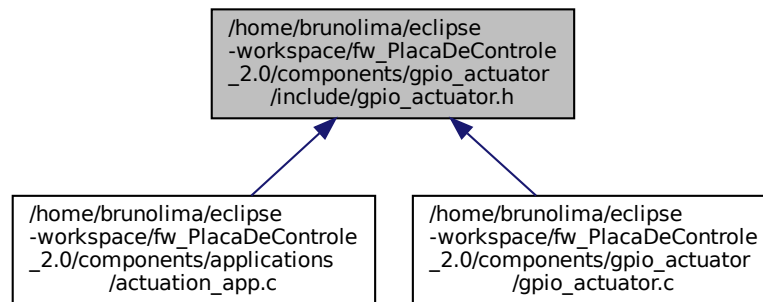
7.10 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/gpio_actuator/include/gpio_actuator.h File Reference

Board GPIO macros and pinout definitions for the GPIO Actuator module.

```
#include "project_config.h"
#include "driver/gpio.h"
Include dependency graph for gpio_actuator.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- #define `GPIO_ACT_SYS_ENABLE` 0
Macro to enable the GPIO actuator system.
- #define `GPIO_ACT_SYS_DISABLE` 1
Macro to disable the GPIO actuator system.
- #define `GPIO_ACT_PERC_FULL_CYCLE` 60000
Macro defining the full cycle duration for the percentimeter (in milliseconds).
- #define `GPIO_ACT_PIN_ON` GPIO_NUM_13
GPIO pin number for turning the system on.
- #define `GPIO_ACT_PIN_OFF` GPIO_NUM_12
GPIO pin number for turning the system off.
- #define `GPIO_ACT_PIN_CW` GPIO_NUM_11
GPIO pin number for clockwise direction.
- #define `GPIO_ACT_PIN_CCW` GPIO_NUM_10
GPIO pin number for counter-clockwise direction.
- #define `GPIO_ACT_PIN_WATERING` GPIO_NUM_9
GPIO pin number for watering relay.
- #define `GPIO_ACT_PIN_AUX` GPIO_NUM_8
GPIO pin number for auxiliary system relay.
- #define `GPIO_ACT_PIN_PERC_AUX` GPIO_NUM_16
GPIO pin number for auxiliary relay for percentimeter.
- #define `GPIO_ACT_PIN_PERC_OUT` GPIO_NUM_35
GPIO pin number for percentimeter reading output.
- #define `GPIO_ACT_PIN_PUMP` GPIO_NUM_14
GPIO pin number for pump relay.
- #define `GPIO_ACT_PIN_CW_IN` GPIO_NUM_7
GPIO pin number for clockwise input.
- #define `GPIO_ACT_PIN_CCW_IN` GPIO_NUM_6
GPIO pin number for counter-clockwise input.
- #define `GPIO_ACT_PIN_PERC_IN` GPIO_NUM_5
GPIO pin number for percentimeter read input.
- #define `GPIO_ACT_PIN_PRESS` GPIO_NUM_4

- *GPIO pin number for water pressure reading input.*
- `#define GPIO_OUTPUT_PIN_GROUP`
GPIO pin bitmask for all output pins.
- `#define GPIO_INPUT_PIN_GROUP`
GPIO pin bitmask for all input pins.
- `#define GPIO_INT_PERC (1ULL<<GPIO_ACT_PIN_PERC_IN)`
GPIO pin bitmask for the percentimeter interrupt pin.

Functions

- `esp_err_t gpio_actuator_init (void)`
Initializes the GPIO actuator module.
- `esp_err_t gpio_actuator_config (pivot_config config)`
Configures the GPIO actuator module with the specified configuration.
- `esp_err_t gpio_actuator_set (pivot_actions actions)`
Sets the pivot configuration.
- `pivot_actions gpio_actuator_get (void)`
Gets the current pivot configuration.
- `void gpio_actuator_shutdown (void)`
Shuts down all relays.
- `void gpio_actuator_pump_on (void)`
Turns on the pump relay.
- `void gpio_actuator_pump_off (void)`
Turns off the pump relay.
- `void gpio_actuator_pressure_on (void)`
Turns on the pressure relay and starts pressure monitoring.
- `void gpio_actuator_pressure_off (void)`
Turns off the pressure relay and stops pressure monitoring.

7.10.1 Detailed Description

Board GPIO macros and pinout definitions for the GPIO Actuator module.

Date

June 20, 2022

7.10.2 Macro Definition Documentation

7.10.2.1 GPIO_ACT_PIN_AUX

```
#define GPIO_ACT_PIN_AUX GPIO_NUM_8
```

GPIO pin number for auxiliary system relay.

System Auxiliary Relay

7.10.2.2 GPIO_ACT_PIN_CCW

```
#define GPIO_ACT_PIN_CCW GPIO_NUM_10
```

GPIO pin number for counter-clockwise direction.

Direction Counter-Clockwise

7.10.2.3 GPIO_ACT_PIN_CCW_IN

```
#define GPIO_ACT_PIN_CCW_IN GPIO_NUM_6
```

GPIO pin number for counter-clockwise input.

Counter-Clockwise Input

7.10.2.4 GPIO_ACT_PIN_CW

```
#define GPIO_ACT_PIN_CW GPIO_NUM_11
```

GPIO pin number for clockwise direction.

Direction Clockwise

7.10.2.5 GPIO_ACT_PIN_CW_IN

```
#define GPIO_ACT_PIN_CW_IN GPIO_NUM_7
```

GPIO pin number for clockwise input.

Clockwise Input

7.10.2.6 GPIO_ACT_PIN_OFF

```
#define GPIO_ACT_PIN_OFF GPIO_NUM_12
```

GPIO pin number for turning the system off.

Main system relay on

7.10.2.7 GPIO_ACT_PIN_ON

```
#define GPIO_ACT_PIN_ON GPIO_NUM_13
```

GPIO pin number for turning the system on.

Main system relay off

7.10.2.8 GPIO_ACT_PIN_PERC_AUX

```
#define GPIO_ACT_PIN_PERC_AUX GPIO_NUM_16
```

GPIO pin number for auxiliary relay for percentimeter.

Auxiliary Relay for Percentimeter

7.10.2.9 GPIO_ACT_PIN_PERC_IN

```
#define GPIO_ACT_PIN_PERC_IN GPIO_NUM_5
```

GPIO pin number for percentimeter read input.

Percentimeter Read Input

7.10.2.10 GPIO_ACT_PIN_PERC_OUT

```
#define GPIO_ACT_PIN_PERC_OUT GPIO_NUM_35
```

GPIO pin number for percentimeter reading output.

Percentimeter

7.10.2.11 GPIO_ACT_PIN_PRESS

```
#define GPIO_ACT_PIN_PRESS GPIO_NUM_4
```

GPIO pin number for water pressure reading input.

Water pressure reading

7.10.2.12 GPIO_ACT_PIN_PUMP

```
#define GPIO_ACT_PIN_PUMP GPIO_NUM_14
```

GPIO pin number for pump relay.

Pump Relay

7.10.2.13 GPIO_ACT_PIN_WATERING

```
#define GPIO_ACT_PIN_WATERING GPIO_NUM_9
```

GPIO pin number for watering relay.

Watering Relay

7.10.2.14 GPIO_INPUT_PIN_GROUP

```
#define GPIO_INPUT_PIN_GROUP
```

Value:

```
( (1ULL<<GPIO_ACT_PIN_CW_IN)      \
  | (1ULL<<GPIO_ACT_PIN_CCW_IN)    \
  | (1ULL<<GPIO_ACT_PIN_PRESS)) \
```

GPIO pin bitmask for all input pins.

7.10.2.15 GPIO_OUTPUT_PIN_GROUP

```
#define GPIO_OUTPUT_PIN_GROUP
```

Value:

```
( (1ULL<<GPIO_ACT_PIN_ON)          \
  | (1ULL<<GPIO_ACT_PIN_OFF)        \
  | (1ULL<<GPIO_ACT_PIN_AUX)        \
  | (1ULL<<GPIO_ACT_PIN_CW)         \
  | (1ULL<<GPIO_ACT_PIN_CCW)        \
  | (1ULL<<GPIO_ACT_PIN_WATERING)   \
  | (1ULL<<GPIO_ACT_PIN_PERC_AUX)   \
  | (1ULL<<GPIO_ACT_PIN_PERC_OUT)   \
  | (1ULL<<GPIO_ACT_PIN_PUMP)) \
```

GPIO pin bitmask for all output pins.

7.10.3 Function Documentation

7.10.3.1 gpio_actuator_config()

```
esp_err_t gpio_actuator_config (  
    pivot_config config )
```

Configures the GPIO actuator module with the specified configuration.

This function configures the GPIO actuator module based on the provided configuration.

Parameters

<i>config</i>	The configuration to apply
---------------	----------------------------

Returns

- ESP_OK: Success
- ESP_FAIL: Failed to configure

Configures the GPIO actuator module with the specified configuration.

This function configures the GPIO actuator module based on the provided configuration.

Parameters

<i>config</i>	Pivot configuration.
---------------	----------------------

Returns

`esp_err_t` Error code indicating the success of the operation.

7.10.3.2 gpio_actuator_get()

```
pivot_actions gpio_actuator_get (  
    void )
```

Gets the current pivot configuration.

This function retrieves the current pivot configuration from the GPIO actuator module.

Returns

The current pivot configuration

Gets the current pivot configuration.

This function retrieves the current GPIO actuator state.

Returns

`pivot_actions` Current pivot actions.

7.10.3.3 gpio_actuator_init()

```
esp_err_t gpio_actuator_init (  
    void )
```

Initializes the GPIO actuator module.

This function initializes the GPIO actuator module, setting up GPIO pins and configurations.

Returns

- `ESP_OK`: Success
- `ESP_FAIL`: Failed to initialize

This function initializes the GPIO actuator module, configures GPIO pins, and sets up timers and tasks.

Returns

`esp_err_t` Error code indicating the success of the operation.

7.10.3.4 gpio_actuator_pressure_off()

```
void gpio_actuator_pressure_off (
    void )
```

Turns off the pressure relay and stops pressure monitoring.

This function turns off the pressure relay and stops pressure monitoring in the GPIO actuator module.

Turns off the pressure relay and stops pressure monitoring.

7.10.3.5 gpio_actuator_pressure_on()

```
void gpio_actuator_pressure_on (
    void )
```

Turns on the pressure relay and starts pressure monitoring.

This function turns on the pressure relay and starts pressure monitoring in the GPIO actuator module.

Turns on the pressure relay and starts pressure monitoring.

7.10.3.6 gpio_actuator_pump_off()

```
void gpio_actuator_pump_off (
    void )
```

Turns off the pump relay.

This function turns off the pump relay in the GPIO actuator module.

Turns off the pump relay.

7.10.3.7 gpio_actuator_pump_on()

```
void gpio_actuator_pump_on (
    void )
```

Turns on the pump relay.

This function turns on the pump relay in the GPIO actuator module.

Turns on the pump relay.

7.10.3.8 gpio_actuator_set()

```
esp_err_t gpio_actuator_set (
    pivot_actions actions )
```

Sets the pivot configuration.

This function sets the pivot configuration for the GPIO actuator module.

Parameters

<i>actions</i>	The pivot actions to set
----------------	--------------------------

Returns

- ESP_OK: Success
- ESP_FAIL: Failed to set the configuration

Sets the pivot configuration.

This function sets the GPIO actuator actions based on the provided actions.

Parameters

<i>actions</i>	Pivot actions to set.
----------------	-----------------------

Returns

`esp_err_t` Error code indicating the success of the operation.

7.10.3.9 gpio_actuator_shutdown()

```
void gpio_actuator_shutdown (
    void )
```

Shuts down all relays.

This function shuts down all relays in the GPIO actuator module.

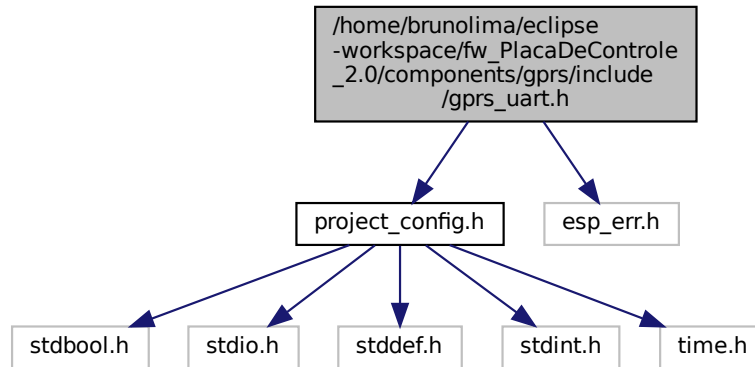
Shuts down all relays.

This function shuts down the GPIO actuator module by turning off relays and cleaning up resources.

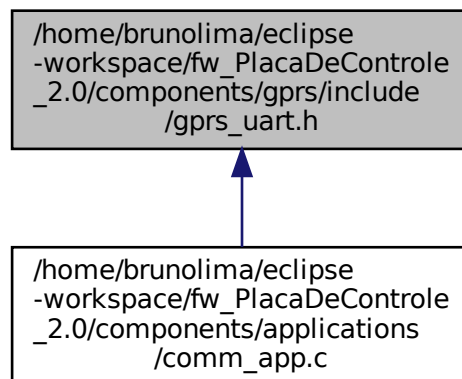
7.11 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_↵ 2.0/components/gprs/include/gprs_uart.h File Reference

Header file for GPRS UART communication functions.

```
#include "project_config.h"
#include "esp_err.h"
Include dependency graph for gprs_uart.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t gprs_uart_init` (const `app_callback` callback)
Initialize the GPRS UART module.
- `esp_err_t gprs_uart_send_event` (const char *event, size_t event_size)
Send an event through the GPRS UART module.

7.11.1 Detailed Description

Header file for GPRS UART communication functions.

7.11.2 Function Documentation

7.11.2.1 gprs_uart_init()

```
esp_err_t gprs_uart_init (
    const app_callback callback )
```

Initialize the GPRS UART module.

This function initializes the GPRS UART module, setting up the UART communication for GPRS.

Parameters

<i>callback</i>	The callback function to be called when data is received.
-----------------	---

Returns

- ESP_OK: Success
- ESP_FAIL: Fail to initialize
- ESP_ERR_INVALID_ARG: Invalid callback function

7.11.2.2 gprs_uart_send_event()

```
esp_err_t gprs_uart_send_event (
    const char * event,
    size_t event_size )
```

Send an event through the GPRS UART module.

This function sends an event buffer through the GPRS UART module.

Parameters

<i>event</i>	The event buffer to be sent.
<i>event_size</i>	The size of the event buffer.

Returns

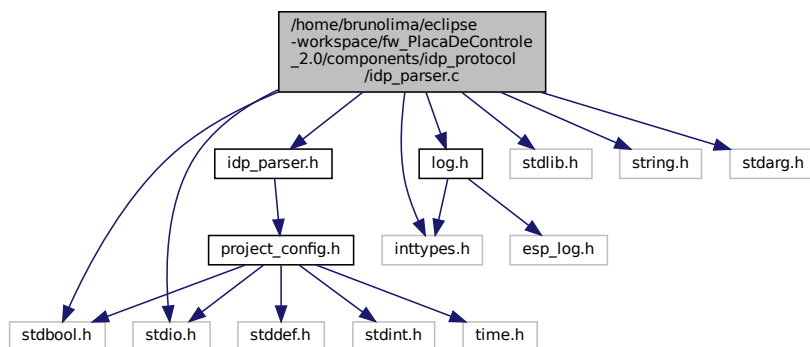
- ESP_OK: Success
- ESP_FAIL: Fail to send the event

7.12 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/idp_parser.c File Reference

Implementation of the IDP (Intelligent Device Protocol) parser.

```
#include "idp_parser.h"
#include "log.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <stdbool.h>
#include <inttypes.h>
```

Include dependency graph for idp_parser.c:



Macros

- `#define IDP_PARSER_TAG "idp_parser"`
The tag used for logging IDP parser-related messages.
- `#define IDP_MAX_PKG_SIZE 200`
The maximum size of the buffer used for creating IDP packets.

Functions

- `idp_type idp_parser_get (const char *string_in, char *string_out)`
Get the IDP type from the given string.
- `bool idp_parser_validate_actions (const pivot_actions actions)`
Validate the actions in a pivot_actions structure.
- `bool idp_parser_validate_network (const network_config net_config)`
Validate the network configuration.

- uint16_t [idp_parser_create_pwd](#) (pivot_actions actions)
Create a password from the actions in a `pivot_actions` structure.
- void [idp_parser_get_pwd](#) (uint16_t pwd, pivot_actions *actions)
Get the `pivot_actions` structure from a password.
- void [idp_parser_create_package](#) (char *str_out, [arg_pair_t](#) arg_pairs[])
Create an IDP package from an array of argument pairs.
- void [idp_parser_get_packet_data](#) (const char *str_arg, [arg_pair_t](#) arg_pairs[])
Get the data from an IDP package string.

7.12.1 Detailed Description

Implementation of the IDP (Intelligent Device Protocol) parser.

This file provides functions to parse IDP packets and create IDP packets with various data types.

7.12.2 Function Documentation

7.12.2.1 idp_parser_create_package()

```
void idp_parser_create_package (
    char * str_out,
    arg\_pair\_t arg_pairs[] )
```

Create an IDP package from an array of argument pairs.

Create an IDP packet with the given arguments.

This function creates an IDP package from an array of argument pairs.

Parameters

out	<i>str_out</i>	The created IDP package string.
in	<i>arg_pairs</i>	The array of argument pairs.

7.12.2.2 idp_parser_create_pwd()

```
uint16_t idp_parser_create_pwd (
    pivot_actions actions )
```

Create a password from the actions in a `pivot_actions` structure.

Create an IDP packet with a generated password based on the specified pivot actions.

This function creates a password from the actions in a `pivot_actions` structure.

Parameters

in	<i>actions</i>	The <code>pivot_actions</code> structure.
----	----------------	---

Returns

The generated password.

7.12.2.3 idp_parser_get()

```
idp_type idp_parser_get (
    const char * string_in,
    char * string_out )
```

Get the IDP type from the given string.

Parse the IDP packet and extract the IDP type.

This function extracts the IDP type from the received IDP packet string.

Parameters

in	<i>string_in</i>	The received IDP packet string.
out	<i>string_out</i>	The extracted IDP type as a string.

Returns

The IDP type as an `idp_type` enum.

7.12.2.4 idp_parser_get_packet_data()

```
void idp_parser_get_packet_data (
    const char * str_arg,
    arg_pair_t arg_pairs[] )
```

Get the data from an IDP package string.

Parse the IDP packet and extract data to populate the argument pairs.

This function extracts data from an IDP package string and fills an array of argument pairs.

Parameters

in	<i>str_arg</i>	The IDP package string.
out	<i>arg_pairs</i>	The array of argument pairs to fill.

7.12.2.5 idp_parser_get_pwd()

```
void idp_parser_get_pwd (
    uint16_t pwd,
    pivot_actions * actions )
```

Get the `pivot_actions` structure from a password.

Get pivot actions from the specified password.

This function extracts the `pivot_actions` structure from a password.

Parameters

in	<i>pwd</i>	The password to extract from.
out	<i>actions</i>	The extracted <code>pivot_actions</code> structure.

7.12.2.6 idp_parser_validate_actions()

```
bool idp_parser_validate_actions (
    const pivot_actions actions )
```

Validate the actions in a `pivot_actions` structure.

Validate the specified pivot actions.

This function validates the actions in a `pivot_actions` structure to ensure they are within valid ranges.

Parameters

in	<i>actions</i>	The <code>pivot_actions</code> structure to validate.
----	----------------	---

Returns

true if the actions are valid, false otherwise.

7.12.2.7 idp_parser_validate_network()

```
bool idp_parser_validate_network (
    const network_config net_config )
```

Validate the network configuration.

Validate the specified network configuration.

This function validates the network configuration to ensure all necessary fields are filled.

Parameters

in	<i>net_config</i>	The network configuration to validate.
----	-------------------	--

Returns

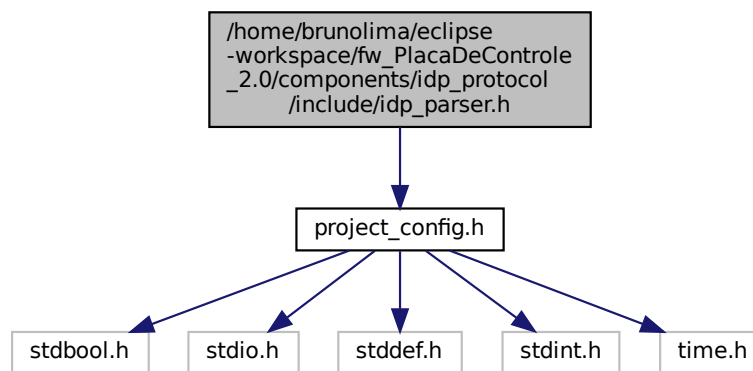
true if the configuration is valid, false otherwise.

7.13 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/idp_protocol/include/idp_parser.h File Reference

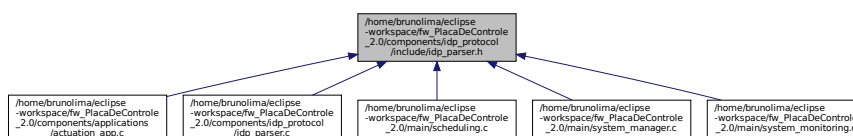
IDP Parser class for handling IDP protocol packets.

```
#include "project_config.h"
```

Include dependency graph for idp_parser.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [arg_pair](#)

Structure representing an argument pair (type and value).

Typedefs

- typedef struct [arg_pair](#) [arg_pair_t](#)

Functions

- [idp_type idp_parser_get](#) (const char *string_in, char *string_out)
Parse the IDP packet and extract the IDP type.
- bool [idp_parser_validate_actions](#) (const pivot_actions actions)
Validate the specified pivot actions.
- bool [idp_parser_validate_network](#) (const network_config net_config)
Validate the specified network configuration.
- uint16_t [idp_parser_create_pwd](#) (pivot_actions actions)
Create an IDP packet with a generated password based on the specified pivot actions.
- void [idp_parser_get_pwd](#) (uint16_t pwd, pivot_actions *actions)
Get pivot actions from the specified password.
- void [idp_parser_create_package](#) (char *str_out, [arg_pair_t](#) arg_pairs[])
Create an IDP packet with the given arguments.
- void [idp_parser_get_packet_data](#) (const char *str_arg, [arg_pair_t](#) arg_pairs[])
Parse the IDP packet and extract data to populate the argument pairs.

7.13.1 Detailed Description

IDP Parser class for handling IDP protocol packets.

Date

July 12, 2023

This file contains the declaration of the IDP Parser class, which is responsible for parsing and handling IDP (Integrated Data Protocol) protocol packets.

7.13.2 Function Documentation

7.13.2.1 idp_parser_create_package()

```
void idp_parser_create_package (
    char * str_out,
    arg\_pair\_t arg_pairs[] )
```

Create an IDP packet with the given arguments.

This function creates an IDP packet string using the provided arguments.

Parameters

out	<i>str_out</i>	The output buffer to store the created IDP packet.
in	<i>arg_pairs</i>	The array of argument pairs (type-value) to be included in the packet.

Create an IDP packet with the given arguments.

This function creates an IDP package from an array of argument pairs.

Parameters

out	<i>str_out</i>	The created IDP package string.
in	<i>arg_pairs</i>	The array of argument pairs.

7.13.2.2 idp_parser_create_pwd()

```
uint16_t idp_parser_create_pwd (  
    pivot_actions actions )
```

Create an IDP packet with a generated password based on the specified pivot actions.

This function creates an IDP packet with a generated password based on the specified pivot actions.

Parameters

in	<i>actions</i>	The pivot actions used to generate the password.
----	----------------	--

Returns

The generated password.

Create an IDP packet with a generated password based on the specified pivot actions.

This function creates a password from the actions in a `pivot_actions` structure.

Parameters

in	<i>actions</i>	The <code>pivot_actions</code> structure.
----	----------------	---

Returns

The generated password.

7.13.2.3 idp_parser_get()

```
idp_type idp_parser_get (
    const char * string_in,
    char * string_out )
```

Parse the IDP packet and extract the IDP type.

This function extracts the IDP type from the received IDP packet.

Parameters

in	<i>string_in</i>	The received IDP packet string.
out	<i>string_out</i>	The IDP type extracted from the packet.

Returns

The IDP type extracted from the packet, or IDP_INVALID if the packet is invalid.

Parse the IDP packet and extract the IDP type.

This function extracts the IDP type from the received IDP packet string.

Parameters

in	<i>string_in</i>	The received IDP packet string.
out	<i>string_out</i>	The extracted IDP type as a string.

Returns

The IDP type as an `idp_type` enum.

7.13.2.4 idp_parser_get_packet_data()

```
void idp_parser_get_packet_data (
    const char * str_arg,
    arg_pair_t arg_pairs[] )
```

Parse the IDP packet and extract data to populate the argument pairs.

This function parses the received IDP packet and extracts data to populate the argument pairs.

Parameters

in	<i>str_arg</i>	The received IDP packet string.
out	<i>arg_pairs</i>	The array of argument pairs to be populated with data from the packet.

Parse the IDP packet and extract data to populate the argument pairs.

This function extracts data from an IDP package string and fills an array of argument pairs.

Parameters

in	<i>str_arg</i>	The IDP package string.
out	<i>arg_pairs</i>	The array of argument pairs to fill.

7.13.2.5 idp_parser_get_pwd()

```
void idp_parser_get_pwd (
    uint16_t pwd,
    pivot_actions * actions )
```

Get pivot actions from the specified password.

This function retrieves pivot actions from the specified password.

Parameters

in	<i>pwd</i>	The password from which to extract pivot actions.
out	<i>actions</i>	The pivot actions extracted from the password.

Get pivot actions from the specified password.

This function extracts the `pivot_actions` structure from a password.

Parameters

in	<i>pwd</i>	The password to extract from.
out	<i>actions</i>	The extracted <code>pivot_actions</code> structure.

7.13.2.6 idp_parser_validate_actions()

```
bool idp_parser_validate_actions (
    const pivot_actions actions )
```

Validate the specified pivot actions.

This function validates the specified pivot actions to ensure they conform to the IDP protocol.

Parameters

in	<i>actions</i>	The pivot actions to be validated.
----	----------------	------------------------------------

Returns

true if the actions are valid, false otherwise.

Validate the specified pivot actions.

This function validates the actions in a `pivot_actions` structure to ensure they are within valid ranges.

Parameters

in	<i>actions</i>	The <code>pivot_actions</code> structure to validate.
----	----------------	---

Returns

true if the actions are valid, false otherwise.

7.13.2.7 idp_parser_validate_network()

```
bool idp_parser_validate_network (
    const network_config net_config )
```

Validate the specified network configuration.

This function validates the specified network configuration to ensure it conforms to the IDP protocol.

Parameters

in	<i>net_config</i>	The network configuration to be validated.
----	-------------------	--

Returns

true if the network configuration is valid, false otherwise.

Validate the specified network configuration.

This function validates the network configuration to ensure all necessary fields are filled.

Parameters

in	<i>net_config</i>	The network configuration to validate.
----	-------------------	--

true if the configuration is valid, false otherwise.

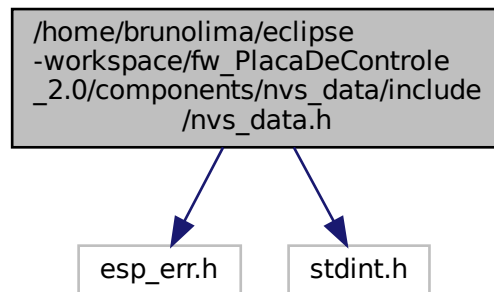
7.14 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/nvs_data/include/nvs_data.h File Reference

Class for direct access to flash memory (NVS).

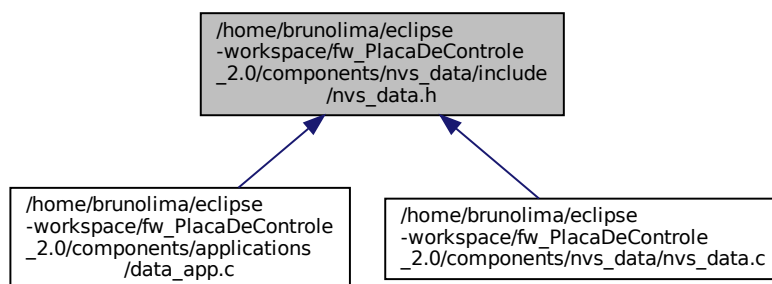
```
#include "esp_err.h"
```

```
#include <stdint.h>
```

Include dependency graph for nvs_data.h:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t nvs_data_init` (void)
Initializes the NVS data class.
- `esp_err_t nvs_data_set` (const char *label_name, const void *value, size_t length)

Sets the value in the NVS data.

- `size_t nvs_data_get_size` (const char *label_name)

Gets the size of the NVS data.

- `esp_err_t nvs_data_get_blob` (const char *label_name, uint8_t *out_value)

Gets the NVS data as a blob.

7.14.1 Detailed Description

Class for direct access to flash memory (NVS).

This file defines the interface for the NVS Data class, which provides functions for direct access to flash memory using the Non-Volatile Storage (NVS) API.

7.14.2 Function Documentation

7.14.2.1 `nvs_data_get_blob()`

```
esp_err_t nvs_data_get_blob (
    const char * label_name,
    uint8_t * out_value )
```

Gets the NVS data as a blob.

This function retrieves the NVS data as a blob under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>out_value</i>	The buffer to store the retrieved data.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

7.14.2.2 `nvs_data_get_size()`

```
size_t nvs_data_get_size (
    const char * label_name )
```

Gets the size of the NVS data.

This function retrieves the size of the NVS data under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
-------------------	----------------------------------

Returns

`size_t` The size of the NVS data.

7.14.2.3 `nvs_data_init()`

```
esp_err_t nvs_data_init (  
    void )
```

Initializes the NVS data class.

This function initializes the NVS data class by initializing the NVS flash memory.

Returns

`esp_err_t` An error code indicating the success or failure of the initialization.

7.14.2.4 `nvs_data_set()`

```
esp_err_t nvs_data_set (  
    const char * label_name,  
    const void * value,  
    size_t length )
```

Sets the value in the NVS data.

This function sets the value in the NVS data under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>value</i>	The value to be stored.
<i>length</i>	The length of the value.

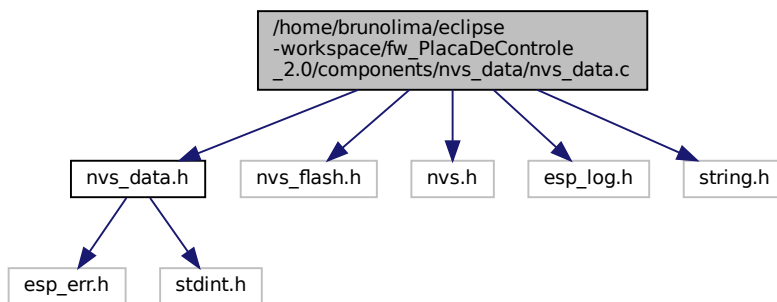
Returns

`esp_err_t` An error code indicating the success or failure of the operation.

7.15 `/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/nvs_data/nvs_data.c` File Reference

Class for direct access to flash memory (NVS).

```
#include "nvs_data.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "esp_log.h"
#include "string.h"
Include dependency graph for nvs_data.c:
```



Macros

- `#define NVS_DATA_TAG "nvs_data"`
Tag used for logging related to NVS data.

Functions

- `esp_err_t nvs_data_init` (void)
Initializes the NVS data class.
- `esp_err_t nvs_data_set` (const char *label_name, const void *value, size_t length)
Sets the value in the NVS data.
- `size_t nvs_data_get_size` (const char *label_name)
Gets the size of the NVS data.
- `esp_err_t nvs_data_get_blob` (const char *label_name, uint8_t *out_value)
Gets the NVS data as a blob.

7.15.1 Detailed Description

Class for direct access to flash memory (NVS).

This file contains the implementation of the NVS Data class, which provides functions for direct access to flash memory using the Non-Volatile Storage (NVS) API.

7.15.2 Function Documentation

7.15.2.1 nvs_data_get_blob()

```
esp_err_t nvs_data_get_blob (
    const char * label_name,
    uint8_t * out_value )
```

Gets the NVS data as a blob.

This function retrieves the NVS data as a blob under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>out_value</i>	The buffer to store the retrieved data.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

7.15.2.2 nvs_data_get_size()

```
size_t nvs_data_get_size (
    const char * label_name )
```

Gets the size of the NVS data.

This function retrieves the size of the NVS data under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
-------------------	----------------------------------

Returns

`size_t` The size of the NVS data.

7.15.2.3 nvs_data_init()

```
esp_err_t nvs_data_init (
    void )
```

Initializes the NVS data class.

This function initializes the NVS data class by initializing the NVS flash memory.

Returns

`esp_err_t` An error code indicating the success or failure of the initialization.

7.15.2.4 nvs_data_set()

```
esp_err_t nvs_data_set (
    const char * label_name,
    const void * value,
    size_t length )
```

Sets the value in the NVS data.

This function sets the value in the NVS data under the specified label and key.

Parameters

<i>label_name</i>	The label name for the NVS data.
<i>value</i>	The value to be stored.
<i>length</i>	The length of the value.

Returns

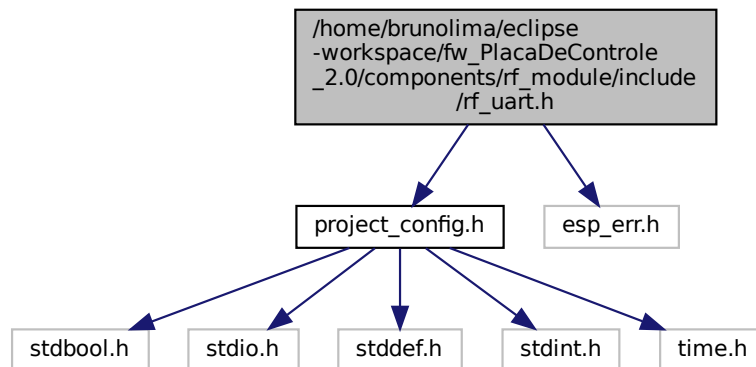
`esp_err_t` An error code indicating the success or failure of the operation.

7.16 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_↵ 2.0/components/rf_module/include/rf_uart.h File Reference

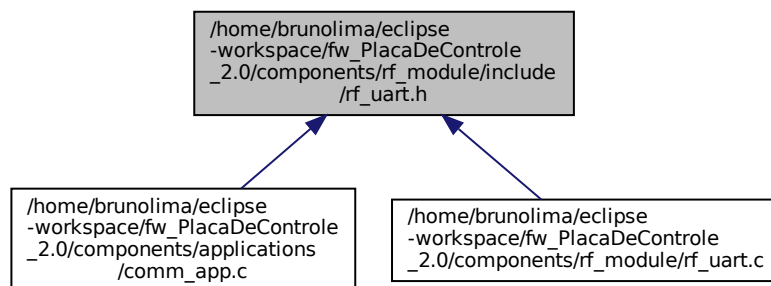
RF UART module functions and callbacks.

```
#include "project_config.h"
#include "esp_err.h"
```

Include dependency graph for rf_uart.h:



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t rf_uart_init` (const `app_callback` callback)
Initialize the RF UART module.
- `esp_err_t rf_uart_send_event` (const char *event, size_t event_size)
Send events through the UART.

7.16.1 Detailed Description

RF UART module functions and callbacks.

Date

June 21, 2022

This file declares functions for initializing and interacting with the RF UART module.

7.16.2 Function Documentation

7.16.2.1 rf_uart_init()

```
esp_err_t rf_uart_init (
    const app_callback callback )
```

Initialize the RF UART module.

This function initializes the RF UART module and sets the callback function for handling received events.

Parameters

<i>callback</i>	Function pointer to the module application class.
-----------------	---

Returns

- ESP_OK: Success
- ESP_FAIL: Fail
- ESP_ERR_INVALID_ARG: Invalid callback

This function configures the UART parameters, installs the UART driver, and creates a task to handle UART events.

Parameters

<i>callback</i>	Callback function for RF UART events.
-----------------	---------------------------------------

Returns

esp_err_t Returns ESP_OK on success, ESP_FAIL on failure.

7.16.2.2 rf_uart_send_event()

```
esp_err_t rf_uart_send_event (
    const char * event,
    size_t event_size )
```

Send events through the UART.

This function sends events through the RF UART module.

Parameters

<i>event</i>	Buffer containing the event data.
<i>event_size</i>	Size of the event buffer.

Returns

- ESP_OK: Success
- ESP_FAIL: Fail

Send events through the UART.

Parameters

<i>event</i>	Pointer to the event data.
<i>event_size</i>	Size of the event data.

Returns

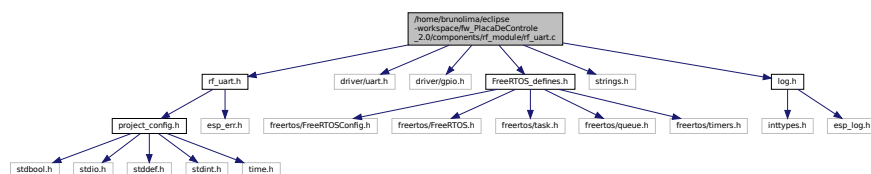
esp_err_t Returns ESP_OK on success, ESP_FAIL on failure.

7.17 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rf_module/rf_uart.c File Reference

RF UART module implementation.

```
#include "rf_uart.h"
#include "driver/uart.h"
#include "driver/gpio.h"
#include "FreeRTOS_defines.h"
#include <strings.h>
#include "log.h"
```

Include dependency graph for rf_uart.c:



Macros

- `#define RF_UART_TAG "rf_uart"`
The log tag for RF UART module.
- `#define RF_UART_NUM UART_NUM_2`
The UART number to use.
- `#define RF_UART_TX_NUM GPIO_NUM_2`
The GPIO number for UART TX.
- `#define RF_UART_RX_NUM GPIO_NUM_1`
The GPIO number for UART RX.
- `#define RF_UART_BUF_SIZE (1024)`
The buffer size for UART communication.

Functions

- `esp_err_t rf_uart_init` (const `app_callback` callback)
Initialize the RF UART module.
- `esp_err_t rf_uart_send_event` (const char *event, size_t event_size)
Send an event over RF UART.

7.17.1 Detailed Description

RF UART module implementation.

This file contains the implementation of the RF UART module for communication over UART. It uses FreeRTOS and ESP-IDF UART driver.

7.17.2 Function Documentation

7.17.2.1 `rf_uart_init()`

```
esp_err_t rf_uart_init (
    const app_callback callback )
```

Initialize the RF UART module.

This function configures the UART parameters, installs the UART driver, and creates a task to handle UART events.

Parameters

<i>callback</i>	Callback function for RF UART events.
-----------------	---------------------------------------

Returns

`esp_err_t` Returns ESP_OK on success, ESP_FAIL on failure.

7.17.2.2 `rf_uart_send_event()`

```
esp_err_t rf_uart_send_event (
    const char * event,
    size_t event_size )
```

Send an event over RF UART.

Send events through the UART.

Parameters

<i>event</i>	Pointer to the event data.
<i>event_size</i>	Size of the event data.

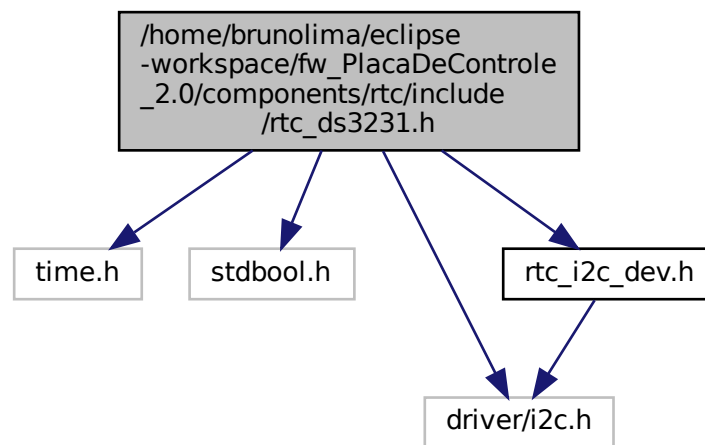
Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_FAIL` on failure.

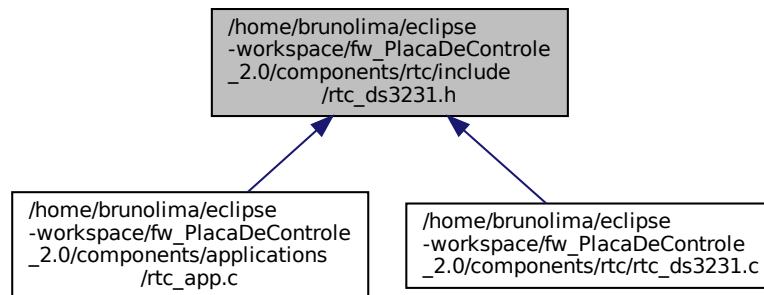
7.18 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/include/rtc_ds3231.h File Reference

RTC DS3231 class for interfacing with the DS3231 RTC module.

```
#include <time.h>
#include <stdbool.h>
#include "driver/i2c.h"
#include "rtc_i2c_dev.h"
Include dependency graph for rtc_ds3231.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define DS3231_ADDR 0x68`
The I2C address of the DS3231 RTC module.
- `#define DS3231_STAT_OSCILLATOR 0x80`
Status bit indicating the oscillator stop flag in the DS3231.
- `#define DS3231_STAT_32KHZ 0x08`
Status bit indicating the 32kHz output flag in the DS3231.
- `#define DS3231_STAT_BUSY 0x04`
Status bit indicating the busy flag in the DS3231.
- `#define DS3231_STAT_ALARM_2 0x02`
Status bit indicating the alarm 2 flag in the DS3231.
- `#define DS3231_STAT_ALARM_1 0x01`
Status bit indicating the alarm 1 flag in the DS3231.
- `#define DS3231_CTRL_OSCILLATOR 0x80`
Control bit for enabling/disabling the oscillator in the DS3231.
- `#define DS3231_CTRL_SQUAREWAVE_BB 0x40`
Control bit for enabling/disabling the square wave output in the DS3231.
- `#define DS3231_CTRL_TEMPCONV 0x20`
Control bit for initiating temperature conversion in the DS3231.
- `#define DS3231_CTRL_ALARM_INTS 0x04`
Control bit for enabling/disabling alarm interrupts in the DS3231.
- `#define DS3231_CTRL_ALARM2_INT 0x02`
Control bit for enabling/disabling alarm 2 interrupt in the DS3231.
- `#define DS3231_CTRL_ALARM1_INT 0x01`
Control bit for enabling/disabling alarm 1 interrupt in the DS3231.
- `#define DS3231_ALARM_WDAY 0x40`
Flag indicating that the alarm is set to match both the date and weekday in the DS3231.
- `#define DS3231_ALARM_NOTSET 0x80`
Flag indicating that the alarm is not set in the DS3231.
- `#define DS3231_ADDR_TIME 0x00`
The address of the time registers in the DS3231.
- `#define DS3231_ADDR_ALARM1 0x07`
The address of the alarm 1 registers in the DS3231.

- `#define DS3231_ADDR_ALARM2 0x0b`
The address of the alarm 2 registers in the DS3231.
- `#define DS3231_ADDR_CONTROL 0x0e`
The address of the control register in the DS3231.
- `#define DS3231_ADDR_STATUS 0x0f`
The address of the status register in the DS3231.
- `#define DS3231_ADDR_AGING 0x10`
The address of the aging offset register in the DS3231.
- `#define DS3231_ADDR_TEMP 0x11`
The address of the temperature registers in the DS3231.
- `#define DS3231_12HOUR_FLAG 0x40`
Flag indicating 12-hour mode in the DS3231.
- `#define DS3231_12HOUR_MASK 0x1f`
Mask for extracting the hour value in 12-hour mode from the DS3231.
- `#define DS3231_PM_FLAG 0x20`
Flag indicating PM (post meridiem) in 12-hour mode in the DS3231.
- `#define DS3231_MONTH_MASK 0x1f`
Mask for extracting the month value from the DS3231.

Functions

- `uint8_t bcd2dec (uint8_t val)`
Converts a binary-coded decimal (BCD) value to decimal.
- `uint8_t dec2bcd (uint8_t val)`
Converts a decimal value to binary-coded decimal (BCD).
- `esp_err_t ds3231_init_desc (rtc_i2c_dev_t *dev, i2c_port_t port, gpio_num_t sda_gpio, gpio_num_t scl_gpio)`
Initializes the DS3231 RTC descriptor.
- `esp_err_t ds3231_set_time (rtc_i2c_dev_t *dev, struct tm *time)`
Sets the time on the DS3231 RTC module.
- `esp_err_t ds3231_get_raw_temp (rtc_i2c_dev_t *dev, int16_t *temp)`
Gets the raw temperature value from the DS3231 RTC module.
- `esp_err_t ds3231_get_temp_integer (rtc_i2c_dev_t *dev, int8_t *temp)`
Gets the integer part of the temperature from the DS3231 RTC module.
- `esp_err_t ds3231_get_temp_float (rtc_i2c_dev_t *dev, float *temp)`
Gets the floating-point representation of the temperature from the DS3231 RTC module.
- `esp_err_t ds3231_get_time (rtc_i2c_dev_t *dev, struct tm *time)`
Gets the current time from the DS3231 RTC module.

7.18.1 Detailed Description

RTC DS3231 class for interfacing with the DS3231 RTC module.

This header file defines the RTC DS3231 class and its associated functions for interfacing with the DS3231 RTC module.

7.18.2 Function Documentation

7.18.2.1 bcd2dec()

```
uint8_t bcd2dec (
    uint8_t val )
```

Converts a binary-coded decimal (BCD) value to decimal.

Parameters

<i>val</i>	The BCD value to convert.
------------	---------------------------

Returns

uint8_t The decimal value.

Converts a binary-coded decimal (BCD) value to decimal.

Parameters

<i>val</i>	Binary-coded decimal value.
------------	-----------------------------

Returns

Decimal value.

7.18.2.2 dec2bcd()

```
uint8_t dec2bcd (
    uint8_t val )
```

Converts a decimal value to binary-coded decimal (BCD).

Parameters

<i>val</i>	The decimal value to convert.
------------	-------------------------------

Returns

uint8_t The BCD value.

Converts a decimal value to binary-coded decimal (BCD).

Parameters

<i>val</i>	Decimal value.
------------	----------------

Returns

Binary-coded decimal value.

7.18.2.3 ds3231_get_raw_temp()

```
esp_err_t ds3231_get_raw_temp (
    rtc_i2c_dev_t * dev,
    int16_t * temp )
```

Gets the raw temperature value from the DS3231 RTC module.

This function retrieves the raw temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Gets the raw temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>temp</i>	Pointer to the variable to store the temperature.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.18.2.4 ds3231_get_temp_float()

```
esp_err_t ds3231_get_temp_float (
    rtc_i2c_dev_t * dev,
    float * temp )
```

Gets the floating-point representation of the temperature from the DS3231 RTC module.

This function retrieves the floating-point representation of the temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Gets the floating-point representation of the temperature from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>temp</i>	Pointer to the variable to store the temperature.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.18.2.5 ds3231_get_temp_integer()

```
esp_err_t ds3231_get_temp_integer (
    rtc_i2c_dev_t * dev,
    int8_t * temp )
```

Gets the integer part of the temperature from the DS3231 RTC module.

This function retrieves the integer part of the temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>temp</i>	The pointer to the variable to store the temperature value.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Gets the integer part of the temperature from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>temp</i>	Pointer to the variable to store the temperature.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.18.2.6 ds3231_get_time()

```
esp_err_t ds3231_get_time (
    rtc_i2c_dev_t * dev,
    struct tm * time )
```

Gets the current time from the DS3231 RTC module.

This function retrieves the current time from the DS3231 RTC module and stores it in the specified time structure.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>time</i>	The pointer to the time structure to store the current time.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Gets the current time from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>time</i>	Pointer to the structure to store the time information.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.18.2.7 ds3231_init_desc()

```
esp_err_t ds3231_init_desc (
    rtc_i2c_dev_t * dev,
    i2c_port_t port,
    gpio_num_t sda_gpio,
    gpio_num_t scl_gpio )
```

Initializes the DS3231 RTC descriptor.

This function initializes the DS3231 RTC descriptor with the specified I2C port and GPIO pin numbers for SDA and SCL.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>port</i>	The I2C port number.
<i>sda_gpio</i>	The GPIO number for the I2C SDA signal.
<i>scl_gpio</i>	The GPIO number for the I2C SCL signal.

Returns

`esp_err_t` An error code indicating the success or failure of the initialization.

Initializes the DS3231 RTC descriptor.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>port</i>	I2C port number.
<i>sda_gpio</i>	GPIO number for SDA.
<i>scl_gpio</i>	GPIO number for SCL.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.18.2.8 ds3231_set_time()

```
esp_err_t ds3231_set_time (
    rtc_i2c_dev_t * dev,
    struct tm * time )
```

Sets the time on the DS3231 RTC module.

This function sets the time on the DS3231 RTC module using the specified time structure.

Parameters

<i>dev</i>	The pointer to the RTC I2C device structure.
<i>time</i>	The pointer to the time structure containing the new time.

Returns

`esp_err_t` An error code indicating the success or failure of the operation.

Sets the time on the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>time</i>	Pointer to the structure containing time information.

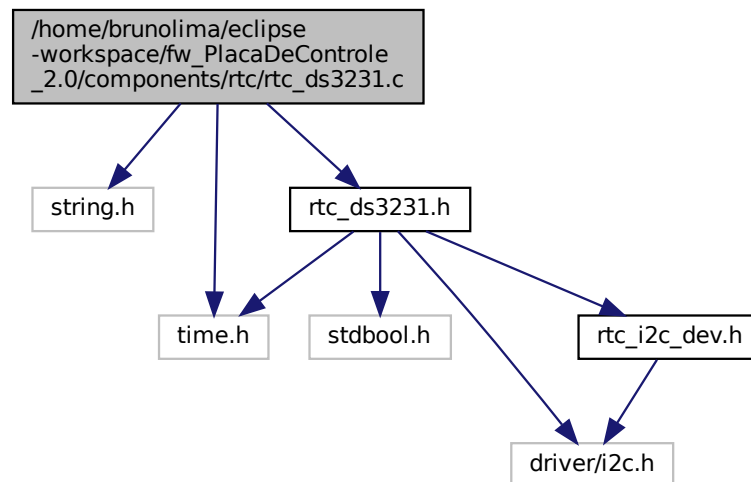
Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.19 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/rtc_ds3231.c File Reference

RTC DS3231 class implementation.

```
#include <string.h>
#include <time.h>
#include "rtc_ds3231.h"
Include dependency graph for rtc_ds3231.c:
```



Macros

- `#define CHECK_ARG(ARG) do { if (!ARG) return ESP_ERR_INVALID_ARG; } while (0)`
Checks if the argument is valid.

Functions

- `uint8_t bcd2dec (uint8_t val)`
Convert binary-coded decimal to decimal.
- `uint8_t dec2bcd (uint8_t val)`
Convert decimal to binary-coded decimal.
- `esp_err_t ds3231_init_desc (rtc_i2c_dev_t *dev, i2c_port_t port, gpio_num_t sda_gpio, gpio_num_t scl_gpio)`
Initialize the DS3231 RTC device.
- `esp_err_t ds3231_set_time (rtc_i2c_dev_t *dev, struct tm *time)`
Set the time on the DS3231 RTC device.
- `esp_err_t ds3231_get_raw_temp (rtc_i2c_dev_t *dev, int16_t *temp)`
Get the raw temperature from the DS3231 RTC device.
- `esp_err_t ds3231_get_temp_integer (rtc_i2c_dev_t *dev, int8_t *temp)`
Get the temperature as an integer from the DS3231 RTC device.
- `esp_err_t ds3231_get_temp_float (rtc_i2c_dev_t *dev, float *temp)`
Get the temperature as a floating-point number from the DS3231 RTC device.
- `esp_err_t ds3231_get_time (rtc_i2c_dev_t *dev, struct tm *time)`
Get the time from the DS3231 RTC device.

7.19.1 Detailed Description

RTC DS3231 class implementation.

This file contains the implementation of the RTC DS3231 class for interfacing with the DS3231 RTC module.

7.19.2 Macro Definition Documentation

7.19.2.1 CHECK_ARG

```
#define CHECK_ARG(  
    ARG ) do { if (!ARG) return ESP_ERR_INVALID_ARG; } while (0)
```

Checks if the argument is valid.

This macro checks if the argument is valid. If the argument is not valid, it returns `ESP_ERR_INVALID_ARG`.

Parameters

<i>ARG</i>	The argument to check.
------------	------------------------

7.19.3 Function Documentation

7.19.3.1 bcd2dec()

```
uint8_t bcd2dec (
    uint8_t val )
```

Convert binary-coded decimal to decimal.

Converts a binary-coded decimal (BCD) value to decimal.

Parameters

<i>val</i>	Binary-coded decimal value.
------------	-----------------------------

Returns

Decimal value.

7.19.3.2 dec2bcd()

```
uint8_t dec2bcd (
    uint8_t val )
```

Convert decimal to binary-coded decimal.

Converts a decimal value to binary-coded decimal (BCD).

Parameters

<i>val</i>	Decimal value.
------------	----------------

Returns

Binary-coded decimal value.

7.19.3.3 ds3231_get_raw_temp()

```
esp_err_t ds3231_get_raw_temp (
    rtc_i2c_dev_t * dev,
    int16_t * temp )
```

Get the raw temperature from the DS3231 RTC device.

Gets the raw temperature value from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>temp</i>	Pointer to the variable to store the temperature.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.19.3.4 ds3231_get_temp_float()

```
esp_err_t ds3231_get_temp_float (
    rtc_i2c_dev_t * dev,
    float * temp )
```

Get the temperature as a floating-point number from the DS3231 RTC device.

Gets the floating-point representation of the temperature from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>temp</i>	Pointer to the variable to store the temperature.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.19.3.5 ds3231_get_temp_integer()

```
esp_err_t ds3231_get_temp_integer (
    rtc_i2c_dev_t * dev,
    int8_t * temp )
```

Get the temperature as an integer from the DS3231 RTC device.

Gets the integer part of the temperature from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>temp</i>	Pointer to the variable to store the temperature.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.19.3.6 ds3231_get_time()

```
esp_err_t ds3231_get_time (
    rtc_i2c_dev_t * dev,
    struct tm * time )
```

Get the time from the DS3231 RTC device.

Gets the current time from the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>time</i>	Pointer to the structure to store the time information.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.19.3.7 ds3231_init_desc()

```
esp_err_t ds3231_init_desc (
    rtc_i2c_dev_t * dev,
    i2c_port_t port,
    gpio_num_t sda_gpio,
    gpio_num_t scl_gpio )
```

Initialize the DS3231 RTC device.

Initializes the DS3231 RTC descriptor.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>port</i>	I2C port number.
<i>sda_gpio</i>	GPIO number for SDA.
<i>scl_gpio</i>	GPIO number for SCL.

Returns

`esp_err_t` Returns `ESP_OK` on success, `ESP_ERR_INVALID_ARG` if arguments are invalid.

7.19.3.8 ds3231_set_time()

```
esp_err_t ds3231_set_time (
    rtc_i2c_dev_t * dev,
    struct tm * time )
```

Set the time on the DS3231 RTC device.

Sets the time on the DS3231 RTC module.

Parameters

<i>dev</i>	Pointer to the RTC device structure.
<i>time</i>	Pointer to the structure containing time information.

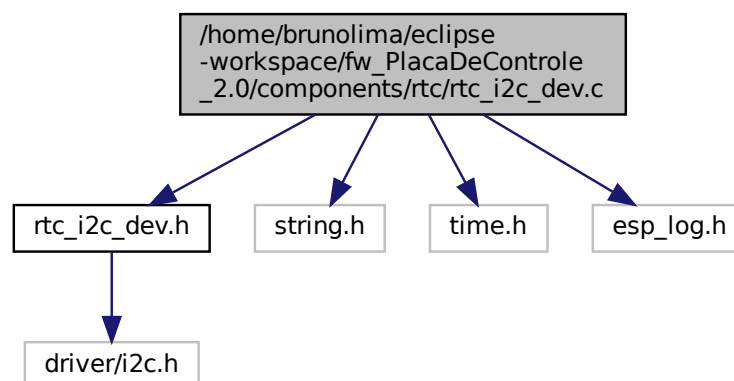
Returns

`esp_err_t` Returns ESP_OK on success, ESP_ERR_INVALID_ARG if arguments are invalid.

7.20 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/rtc/rtc_i2c_dev.c File Reference

RTC I2C device implementation.

```
#include "rtc_i2c_dev.h"
#include <string.h>
#include <time.h>
#include "esp_log.h"
Include dependency graph for rtc_i2c_dev.c:
```



Macros

- `#define RTC_I2C_DEV_TAG "I2CDEV"`

The RTC_I2C_DEV_TAG used for logging related to the RTC I2C device.

Functions

- `esp_err_t i2c_master_init` (`i2c_port_t` port, `int` sda, `int` scl)
Initialize the I2C master.
- `esp_err_t i2c_dev_read` (`const rtc_i2c_dev_t` *dev, `const void` *out_data, `size_t` out_size, `void` *in_data, `size_t` in_size)
Read data from the I2C device.
- `esp_err_t i2c_dev_write` (`const rtc_i2c_dev_t` *dev, `const void` *out_reg, `size_t` out_reg_size, `const void` *out_data, `size_t` out_size)
Write data to the I2C device.

7.20.1 Detailed Description

RTC I2C device implementation.

This file contains the implementation of the RTC I2C device.

7.20.2 Function Documentation

7.20.2.1 i2c_dev_read()

```
esp_err_t i2c_dev_read (  
    const rtc_i2c_dev_t * dev,  
    const void * out_data,  
    size_t out_size,  
    void * in_data,  
    size_t in_size )
```

Read data from the I2C device.

Reads data from the I2C device.

This function reads data from the I2C device.

Parameters

<i>dev</i>	Pointer to the RTC I2C device structure.
<i>out_data</i>	Data to be written before reading (can be NULL).
<i>out_size</i>	Size of the data to be written.
<i>in_data</i>	Pointer to the buffer for reading data.
<i>in_size</i>	Size of the buffer for reading data.

Returns

ESP_OK on success, ESP_ERR_INVALID_ARG if arguments are invalid, ESP_FAIL on failure.

7.20.2.2 i2c_dev_write()

```
esp_err_t i2c_dev_write (
    const rtc_i2c_dev_t * dev,
    const void * out_reg,
    size_t out_reg_size,
    const void * out_data,
    size_t out_size )
```

Write data to the I2C device.

Writes data to the I2C device.

This function writes data to the I2C device.

Parameters

<i>dev</i>	Pointer to the RTC I2C device structure.
<i>out_reg</i>	Register address to write to (can be NULL).
<i>out_reg_size</i>	Size of the register address.
<i>out_data</i>	Data to be written.
<i>out_size</i>	Size of the data to be written.

Returns

ESP_OK on success, ESP_ERR_INVALID_ARG if arguments are invalid, ESP_FAIL on failure.

7.20.2.3 i2c_master_init()

```
esp_err_t i2c_master_init (
    i2c_port_t port,
    int sda,
    int scl )
```

Initialize the I2C master.

Initializes the I2C master for the specified port.

This function initializes the I2C master with the specified parameters.

Parameters

<i>port</i>	I2C port number.
<i>sda</i>	GPIO number for SDA.
<i>scl</i>	GPIO number for SCL.

Returns

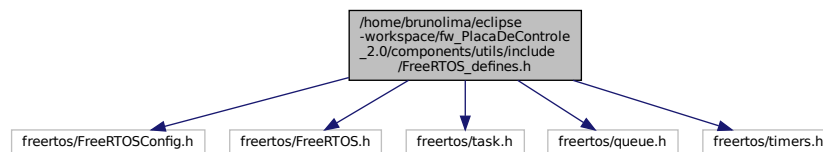
ESP_OK on success, ESP_FAIL on failure.

7.21 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_↵ 2.0/components/utils/include/FreeRTOS_defines.h File Reference

Defines for FreeRTOS tasks and priorities.

```
#include "freertos/FreeRTOSConfig.h"  
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
#include "freertos/queue.h"  
#include "freertos/timers.h"
```

Include dependency graph for FreeRTOS_defines.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define SECTORIZATION_TASK_NAME` "sectorization task"
Name of the main sectorization task.
- `#define SECTORIZATION_TASK_SIZE` (configMINIMAL_STACK_SIZE * 4)
Stack size of the main sectorization task.
- `#define SECTORIZATION_TASK_PRIORITY` (tskIDLE_PRIORITY + 4)
Priority of the main sectorization task.
- `#define MAIN_APP_TASK_2_NAME` "peak hours task"
Name of the main peak hours task.
- `#define MAIN_APP_STACK_2_SIZE` (configMINIMAL_STACK_SIZE * 4)
Stack size of the main peak hours task.
- `#define MAIN_APP_TASK_2_PRIORITY` (tskIDLE_PRIORITY + 3)
Priority of the main peak hours task.
- `#define SCHEDULING_TASK_NAME` "scheduling task"
Name of the main scheduling task.
- `#define SCHEDULING_TASK_SIZE` (configMINIMAL_STACK_SIZE * 6)
Stack size of the main scheduling task.
- `#define SCHEDULING_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)
Priority of the main scheduling task.
- `#define DATA_APP_TASK_NAME` "data app task"
Name of the data app task.

- `#define DATA_APP_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)
Stack size of the data app task.
- `#define DATA_APP_TASK_PRIORITY` (tskIDLE_PRIORITY + 2)
Priority of the data app task.
- `#define COMM_APP_TASK_NAME` "comm app task"
Name of the comm app task.
- `#define COMM_APP_STACK_SIZE` (configMINIMAL_STACK_SIZE * 10)
Stack size of the comm app task.
- `#define COMM_APP_TASK_PRIORITY` (tskIDLE_PRIORITY + 2)
Priority of the comm app task.
- `#define ACTUATION_APP_TASK_NAME` "actuation app task"
Name of the actuation app task.
- `#define ACTUATION_APP_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)
Stack size of the actuation app task.
- `#define ACTUATION_APP_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)
Priority of the actuation app task.
- `#define ACTUATOR_CHECK_TASK_NAME` "actuator check task"
Name of the actuator check task.
- `#define ACTUATOR_CHECK_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)
Stack size of the actuator check task.
- `#define ACTUATOR_CHECK_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)
Priority of the actuator check task.
- `#define ACTUATOR_PERCENT_TASK_NAME` "actuator percent task"
Name of the actuator percent task.
- `#define ACTUATOR_PERCENT_STACK_SIZE` (configMINIMAL_STACK_SIZE * 6)
Stack size of the actuator percent task.
- `#define ACTUATOR_PERCENT_TASK_PRIORITY` (tskIDLE_PRIORITY + 3)
Priority of the actuator percent task.
- `#define GPRS_UART_TASK_NAME` "GPRS UART task"
Name of the GPRS UART task.
- `#define GPRS_UART_STACK_SIZE` (configMINIMAL_STACK_SIZE * 10)
Stack size of the GPRS UART task.
- `#define GPRS_UART_TASK_PRIORITY` (tskIDLE_PRIORITY + 12)
Priority of the GPRS UART task.
- `#define RF_UART_TASK_NAME` "RF UART task"
Name of the RF UART task.
- `#define RF_UART_STACK_SIZE` (configMINIMAL_STACK_SIZE * 10)
Stack size of the RF UART task.
- `#define RF_UART_TASK_PRIORITY` (tskIDLE_PRIORITY + 12)
Priority of the RF UART task.
- `#define WIFI_APP_TASK_NAME` "wifi app task"
Name of the Wi-Fi app task.
- `#define WIFI_APP_STACK_SIZE` (configMINIMAL_STACK_SIZE * 4)
Stack size of the Wi-Fi app task.
- `#define WIFI_APP_TASK_PRIORITY` (tskIDLE_PRIORITY + 8)
Priority of the Wi-Fi app task.
- `#define SYSTEM_MONITORING_TASK_NAME` "monitoring task"
Name of the system monitoring task.
- `#define SYSTEM_MONITORING_TASK_SIZE` (configMINIMAL_STACK_SIZE * 6)
Stack size of the system monitoring task.
- `#define SYSTEM_MONITORING_TASK_PRIORITY` (tskIDLE_PRIORITY + 2)

- Priority of the system monitoring task.*
 - #define ECO_MODE_TASK_NAME "eco mode task"
Name of the Eco Mode task.
 - #define ECO_MODE_TASK_SIZE (configMINIMAL_STACK_SIZE * 4)
Stack size of the Eco Mode task.
 - #define ECO_MODE_TASK_PRIORITY (tskIDLE_PRIORITY + 2)
Priority of the Eco Mode task.

7.21.1 Detailed Description

Defines for FreeRTOS tasks and priorities.

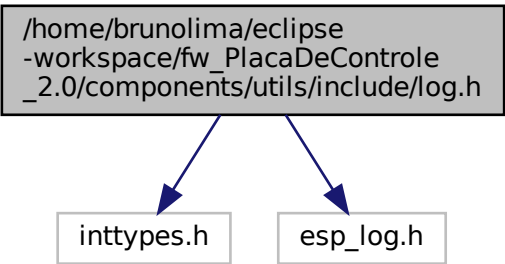
Date
July 11, 2023

This file contains defines for FreeRTOS tasks and priorities used in the application.

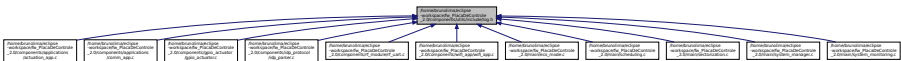
7.22 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utils/include/log.h File Reference

Creating customizable logs.

```
#include <inttypes.h>
#include "esp_log.h"
Include dependency graph for log.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define LOG_COLOR_WHITE "37"`
ANSI color code for white.
- `#define LOG_COMM(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_CYAN) tag, __VA_ARGS__)`
Communication logs.
- `#define LOG_DATA(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_WHITE) tag, __VA_ARGS__)`
Data logs.
- `#define LOG_ACTUATION(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_PURPLE) tag, __VA_ARGS__)`
Actuation logs.
- `#define LOG_MANAGER(tag, ...) ESP_LOGI(LOG_COLOR(LOG_COLOR_GREEN) tag, __VA_ARGS__)`
System manager logs.

7.22.1 Detailed Description

Creating customizable logs.

Date

June 15, 2022

This file contains macros for creating customizable logs.

7.22.2 Macro Definition Documentation

7.22.2.1 LOG_ACTUATION

```
#define LOG_ACTUATION(  
    tag,  
    ... ) ESP_LOGI (LOG_COLOR (LOG_COLOR_PURPLE) tag, __VA_ARGS__)
```

Actuation logs.

This macro is used to create actuation logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
<i>...</i>	The message or additional parameters for the log message.

7.22.2.2 LOG_COLOR_WHITE

```
#define LOG_COLOR_WHITE "37"
```

ANSI color code for white.

This define specifies the ANSI color code for white, which can be used to customize log colors.

7.22.2.3 LOG_COMM

```
#define LOG_COMM(  
    tag,  
    ... ) ESP_LOGI(LOG_COLOR(LOG_COLOR_CYAN) tag, __VA_ARGS__)
```

Communication logs.

This macro is used to create communication logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

7.22.2.4 LOG_DATA

```
#define LOG_DATA(  
    tag,  
    ... ) ESP_LOGI(LOG_COLOR(LOG_COLOR_WHITE) tag, __VA_ARGS__)
```

Data logs.

This macro is used to create data logs with a customizable tag and message.

Parameters

<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

7.22.2.5 LOG_MANAGER

```
#define LOG_MANAGER(  
    tag,  
    ... ) ESP_LOGI(LOG_COLOR(LOG_COLOR_GREEN) tag, __VA_ARGS__)
```

System manager logs.

This macro is used to create actuation logs with a customizable tag and message.

Parameters

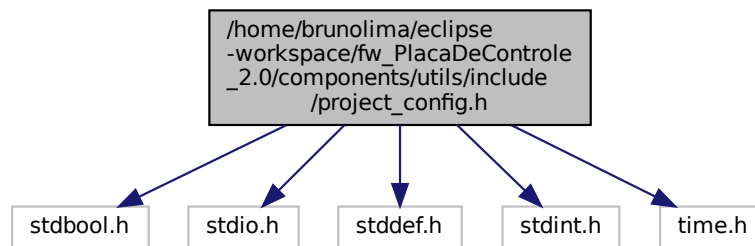
<i>tag</i>	The tag for the log message.
...	The message or additional parameters for the log message.

7.23 `/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/utis/include/project_config.h` File Reference

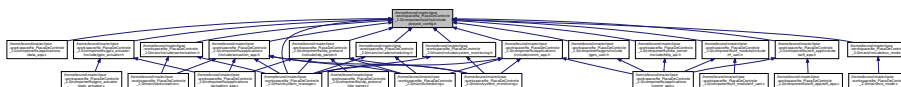
Configuration and parameter definitions for the project.

```
#include <stdbool.h>
#include <stdio.h>
#include <stddef.h>
#include <stdint.h>
#include <time.h>
```

Include dependency graph for `project_config.h`:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `pivot_sectors`
Configuration angles.

Macros

- #define **UNUSED**(x) (void)(sizeof(x))
- #define **CONFIG_FW_VERSION** ("v2.0.0")
Firmware version.
- #define **CONFIG_SCHEDULING_MAX_VALUE** (10)
Maximum number of scheduling values.
- #define **CONFIG_HISTORY_MAX_VALUE** (20)
Maximum number of history values.
- #define **CONFIG_SECTORS_MAX_VALUE** (04)
Maximum number of sectors.
- #define **CONFIG_ACTIONS_UNDEF_VALUE** (655)
Undefined value for actions.
- #define **CONFIG_HTTP_ERROR** ("400")
HTTP error response code.
- #define **CONFIG_HTTP_OK** ("200")
HTTP OK response code.

Typedefs

- typedef void(* **app_callback**) (const char *buffer_request, **comm_type** communication)
Application callback function.

Enumerations

- enum **idp_type** {
 IDP_0 = 0 , **IDP_1** , **IDP_2** , **IDP_3** ,
 IDP_4 , **IDP_5** , **IDP_6** , **IDP_7** ,
 IDP_8 , **IDP_9** , **IDP_10** , **IDP_11** ,
 IDP_12 , **IDP_13** , **IDP_14** , **IDP_15** ,
 IDP_16 , **IDP_17** , **IDP_18** , **IDP_22** = 22 ,
 IDP_30 = 30 , **IDP_90** = 90 , **IDP_91** , **IDP_92** ,
 IDP_INVALID = 255 }
Enumerates different packet identifiers for communication.
- enum **comm_type** { **COMM_HTTP_POST** = 0 , **COMM_HTTP_GET** , **COMM_MQTT** }
Enumerates different communication types.
- enum **pivot_states** {
 PIVOT_ON = 1 , **PIVOT_OFF** = 2 , **PIVOT_CW** = 3 , **PIVOT_CCW** = 4 ,
 PIVOT_DRY = 5 , **PIVOT_WET** = 6 , **PIVOT_PRESSURIZING** = 7 , **PIVOT_UNKNOWN** = 8 }
Pivot actions states.

Functions

- struct **__attribute__** ((__packed__))
Pivot actions parameters.

Variables

- `pivot_actions`
- `pivot_config`
- `network_config`
- `eco_mode_config`
- `sector_config`
- `pivot_return_config`
- `pivot_scheduling_date`
- `pivot_scheduling_off_date`
- `pivot_scheduling_angle`
- `pivot_scheduling_off_angle`
- `pivot_history`
- `uint16_t global_angle`

Global angle variable.

7.23.1 Detailed Description

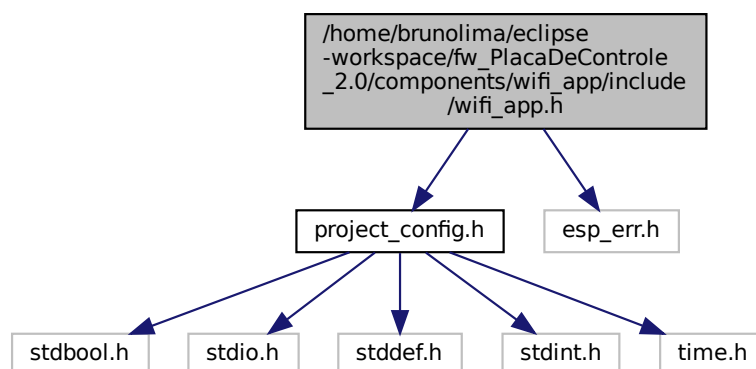
Configuration and parameter definitions for the project.

This file contains configuration and parameter definitions for the project.

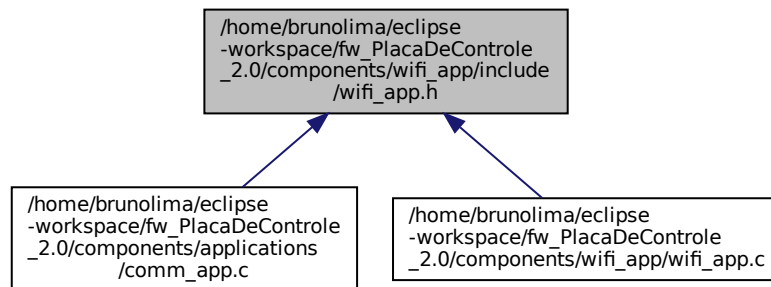
7.24 `/home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/components/wifi_app/include/wifi_app.h` File Reference

Wi-Fi application functions and callbacks.

```
#include "project_config.h"
#include "esp_err.h"
Include dependency graph for wifi_app.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- `esp_err_t wifi_app_init (void)`
Initialize the Wi-Fi application.
- `void wifi_app_set_config (char *wifi_ssid, char *wifi_pass)`
Set Wi-Fi configuration parameters.

7.24.1 Detailed Description

Wi-Fi application functions and callbacks.

Date

January 29, 2023

This file contains the Wi-Fi application initialization and callback registration functions.

7.24.2 Function Documentation

7.24.2.1 wifi_app_init()

```
esp_err_t wifi_app_init (  
    void )
```

Initialize the Wi-Fi application.

This function initializes the Wi-Fi application, including setting up Wi-Fi configurations.

Returns

`esp_err_t` Returns `ESP_OK` on success, an error code otherwise.
`esp_err_t` indicating success or failure.

- *Wi-Fi application tag for logging.*
• #define `WIFI_DEFAULT_IP` "192.168.0.1"
Default IP address for the Wi-Fi access point.
- #define `WIFI_DEFAULT_MASK` "255.255.255.0"
Default subnet mask for the Wi-Fi access point.
- #define `WIFI_CHANNEL` 7
Wi-Fi channel to use for the access point.
- #define `WIFI_MAX_STA_CONN` 5
Maximum number of stations that can be connected to the access point.
- #define `WIFI_APP_SIZE_QUEUE_EVENT` 5

Functions

- `esp_err_t wifi_app_start` (void)
Start the Wi-Fi application.
- `esp_err_t wifi_app_init` (void)
Initialize the Wi-Fi application.
- `void wifi_app_set_config` (char *wifi_ssid, char *wifi_pass)
Set the Wi-Fi configuration.

7.25.1 Detailed Description

Wi-Fi application implementation.

Date

January 20, 2023

This file contains the implementation of the Wi-Fi application. It includes functions to start the Wi-Fi application, handle Wi-Fi events, and manage the Wi-Fi application task.

7.25.2 Function Documentation

7.25.2.1 `wifi_app_init()`

```
esp_err_t wifi_app_init (  
    void )
```

Initialize the Wi-Fi application.

Returns

`esp_err_t` indicating success or failure.

7.25.2.2 `wifi_app_set_config()`

```
void wifi_app_set_config (  
    char * wifi_ssid,  
    char * wifi_pass )
```

Set the Wi-Fi configuration.

Set Wi-Fi configuration parameters.

Parameters

<i>wifi_ssid</i>	Wi-Fi SSID.
<i>wifi_pass</i>	Wi-Fi password.

7.25.2.3 wifi_app_start()

```
esp_err_t wifi_app_start (
    void )
```

Start the Wi-Fi application.

Returns

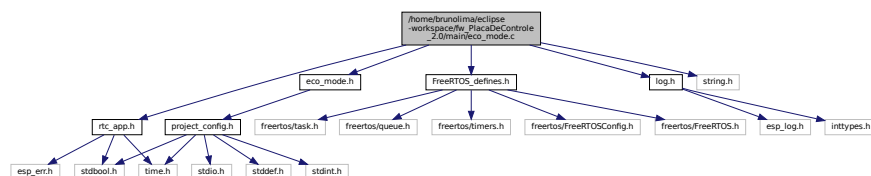
esp_err_t indicating success or failure.

7.26 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/eco_mode.c File Reference

Implementation of the Eco Mode module.

```
#include "eco_mode.h"
#include "rtc_app.h"
#include "FreeRTOS_defines.h"
#include "log.h"
#include <string.h>
```

Include dependency graph for eco_mode.c:



Macros

- #define ECO_MODE_TAG "eco_mode"

Functions

- void [eco_mode_start](#) (eco_mode_config current_eco_mode)
Starts the Eco Mode with the provided configuration.
- void [eco_mode_stop](#) (void)
Stops the Eco Mode.
- void [eco_mode_register_callback](#) (const [app_callback](#) callback)
Registers a callback function for Eco Mode events.

7.26.1 Detailed Description

Implementation of the Eco Mode module.

Author

soil-dev

Date

15 de out. de 2023

7.26.2 Function Documentation

7.26.2.1 `eco_mode_register_callback()`

```
void eco_mode_register_callback (
    const app_callback callback )
```

Registers a callback function for Eco Mode events.

Parameters

<i>callback</i>	Callback function to be registered.
-----------------	-------------------------------------

7.26.2.2 `eco_mode_start()`

```
void eco_mode_start (
    eco_mode_config current_eco_mode )
```

Starts the Eco Mode with the provided configuration.

Parameters

<i>current_eco_mode</i>	Configuration for Eco Mode.
-------------------------	-----------------------------

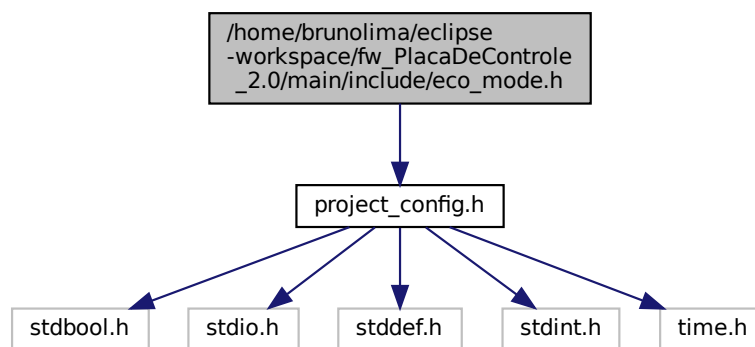
Note

If the start or end time is not set, the Eco Mode will be stopped.

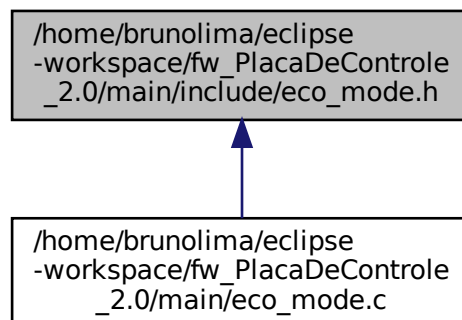
7.27 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/eco_mode.h File Reference

Header file for the Eco Mode module.

```
#include "project_config.h"
Include dependency graph for eco_mode.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [eco_mode_start](#) (eco_mode_config current_eco_mode)
Starts the Eco Mode with the provided configuration.
- void [eco_mode_stop](#) (void)
Stops the Eco Mode.
- void [eco_mode_register_callback](#) (const [app_callback](#) callback)
Registers a callback function for Eco Mode events.

7.27.1 Detailed Description

Header file for the Eco Mode module.

Author

soil-dev

Date

15 de out. de 2023

7.27.2 Function Documentation

7.27.2.1 [eco_mode_register_callback\(\)](#)

```
void eco_mode_register_callback (  
    const app\_callback callback )
```

Registers a callback function for Eco Mode events.

Parameters

<i>callback</i>	Callback function to be registered.
-----------------	-------------------------------------

7.27.2.2 [eco_mode_start\(\)](#)

```
void eco_mode_start (  
    eco_mode_config current_eco_mode )
```

Starts the Eco Mode with the provided configuration.

Parameters

<code>current_eco_mode</code>	Configuration for Eco Mode.
-------------------------------	-----------------------------

Note

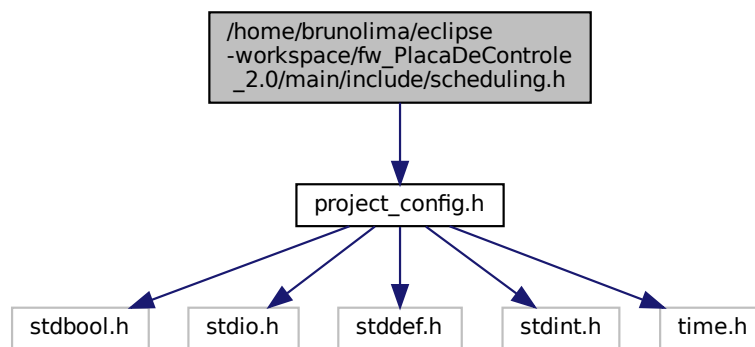
If the start or end time is not set, the Eco Mode will be stopped.

7.28 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/scheduling.h File Reference

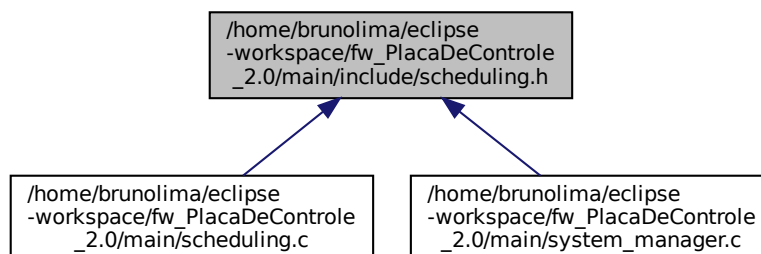
Header file for scheduling functionality.

```
#include "project_config.h"
```

Include dependency graph for scheduling.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `scheduling_start` (`idp_type` scheduling_idp, void *scheduling_data)
Starts a scheduling task based on the provided IDP and data.
- void `scheduling_register_callback` (const `app_callback` callback)
Registers a callback function for scheduling events.

7.28.1 Detailed Description

Header file for scheduling functionality.

Author

soil-dev

Date

31 de jul. de 2023

7.28.2 Function Documentation

7.28.2.1 `scheduling_register_callback()`

```
void scheduling_register_callback (  
    const app_callback callback )
```

Registers a callback function for scheduling events.

This function allows the registration of a callback function that will be invoked for scheduling events. The callback function is used to communicate with other modules or external systems.

Parameters

<i>callback</i>	The callback function to be registered.
-----------------	---

Note

The function checks for null values in the callback parameter.

7.28.2.2 `scheduling_start()`

```
void scheduling_start (  
    idp_type scheduling_idp,  
    void * scheduling_data )
```

Starts a scheduling task based on the provided IDP and data.

This function initiates a scheduling task based on the given IDP (InterDevice Protocol) and the associated data. The task is selected according to the IDP, and the data is used to configure and execute the task.

Parameters

<i>scheduling_idp</i>	The InterDevice Protocol identifier for scheduling.
<i>scheduling_data</i>	Pointer to the data structure associated with the scheduling task.

Note

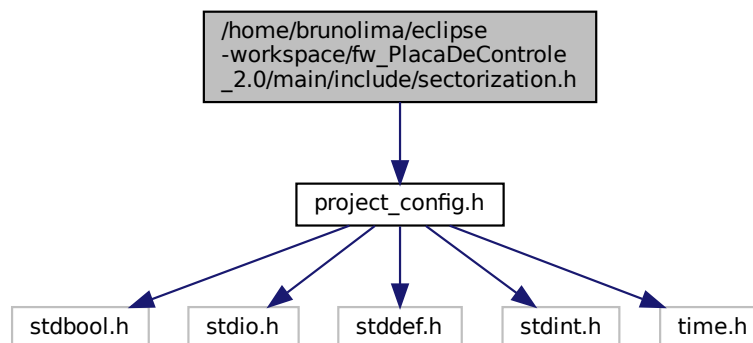
The function checks for null values in the `scheduling_data` parameter and logs an error if found.

7.29 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/sectorization.h File Reference

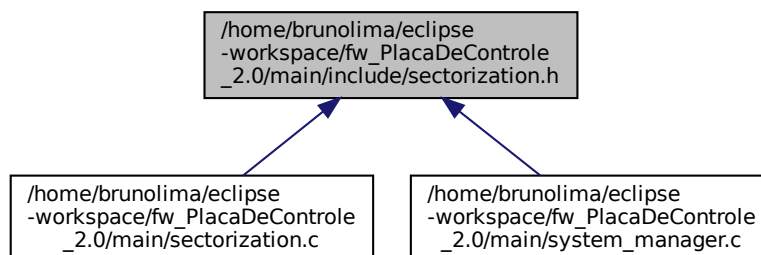
Header file for sectorization functionality.

```
#include "project_config.h"
```

Include dependency graph for sectorization.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [sectorization_start](#) (sector_config sectors)
Starts the sectorization task with the provided sector configuration.
- void [sectorization_stop](#) (void)
Stops the sectorization task.

7.29.1 Detailed Description

Header file for sectorization functionality.

Author

soil-dev

Date

31 de jul. de 2023

7.29.2 Function Documentation

7.29.2.1 [sectorization_start\(\)](#)

```
void sectorization_start (  
    sector_config sectors )
```

Starts the sectorization task with the provided sector configuration.

This function initializes the sectorization task with the specified sector configuration.

Parameters

sectors	Configuration for sectorization.
-------------------------	----------------------------------

7.29.2.2 [sectorization_stop\(\)](#)

```
void sectorization_stop (  
    void )
```

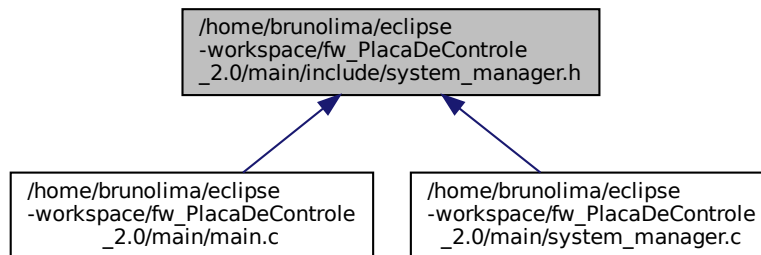
Stops the sectorization task.

This function stops the sectorization task.

7.30 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/system_manager.h File Reference

Header file for system manager module.

This graph shows which files directly or indirectly include this file:



Functions

- void [system_manager_init](#) (void)
Initializes the system manager.

7.30.1 Detailed Description

Header file for system manager module.

Author

Bruno

Date

17 de jul. de 2023

7.30.2 Function Documentation

7.30.2.1 system_manager_init()

```
void system_manager_init (
    void )
```

Initializes the system manager.

This function initializes the system manager module.

Initializes the system manager.

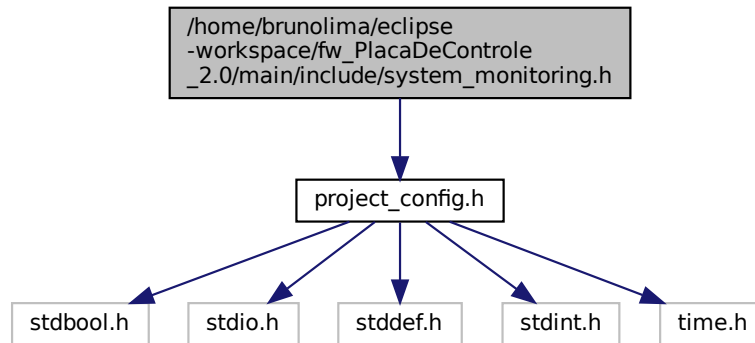
This function initializes various components of the system, including RTC, data storage, actuation, communication, sectorization, and scheduling. It also triggers an automatic reboot based on certain conditions.

7.31 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/include/system_monitoring.h File Reference

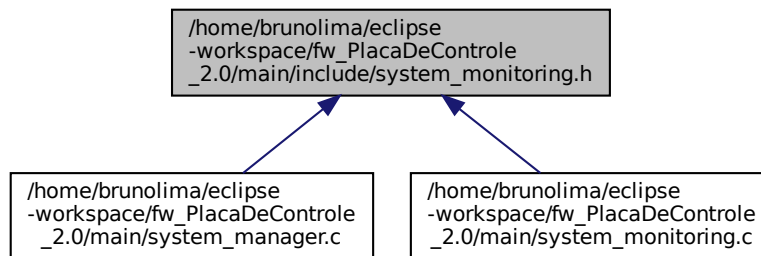
Header file for the system monitoring module.

```
#include "project_config.h"
```

Include dependency graph for system_monitoring.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [system_monitoring_start](#) (pivot_return_config return_config, uint8_t monitoring_time)
Starts the system monitoring with the specified configuration.
- void [system_monitoring_stop](#) (void)
Stops the system monitoring.
- void [system_monitoring_register_callback](#) (const [app_callback](#) callback)
Registers a callback function for system monitoring.

7.31.1 Detailed Description

Header file for the system monitoring module.

This module provides functions to start and stop system monitoring and register a callback function.

Date

4 de ago. de 2023

Author

soil-dev

7.31.2 Function Documentation

7.31.2.1 `system_monitoring_register_callback()`

```
void system_monitoring_register_callback (
    const app_callback callback )
```

Registers a callback function for system monitoring.

This function allows the registration of a callback function to be used in the system monitoring module. The registered callback will be invoked by the system monitoring module to perform specific actions.

Parameters

in	<i>callback</i>	A function pointer to the callback function.
----	-----------------	--

7.31.2.2 `system_monitoring_start()`

```
void system_monitoring_start (
    const pivot_return_config return_config,
    uint8_t monitoring_time )
```

Starts the system monitoring with the specified configuration.

This function initiates the system monitoring module with the provided configuration. It sets up a task to monitor the system based on the specified return configuration and monitoring time.

Parameters

in	<i>return_config</i>	The configuration for pivot return, including start and end angles.
in	<i>monitoring_time</i>	The time interval (in minutes) for monitoring the system.

Starts the system monitoring with the specified configuration.

This function initializes the system monitoring task with the specified configuration.

Parameters

<i>return_config</i>	Configuration for system monitoring.
<i>monitoring_time</i>	Time interval for system monitoring (in minutes).

7.31.2.3 system_monitoring_stop()

```
void system_monitoring_stop (
    void )
```

Stops the system monitoring.

This function stops the system monitoring task and cleans up associated resources.

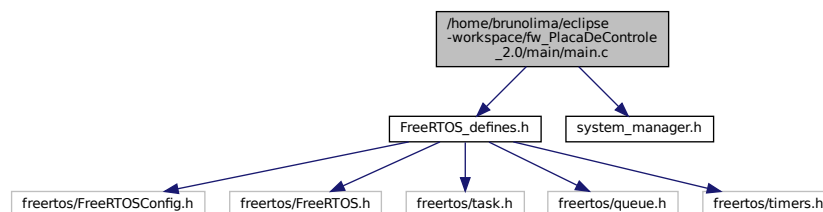
Stops the system monitoring.

This function stops the system monitoring task.

7.32 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/main.c File Reference

Main source file for the application.

```
#include "FreeRTOS_defines.h"
#include "system_manager.h"
Include dependency graph for main.c:
```



Functions

- void `app_main` (void)
Main function of the application.

7.32.1 Detailed Description

Main source file for the application.

Author

brunolima

Date

31 de mai de 2022

7.32.2 Function Documentation

7.32.2.1 app_main()

```
void app_main (
    void )
```

Main function of the application.

This is the main entry point of the application. It initializes the system manager and enters a loop with a delay of 2000 milliseconds.

Note

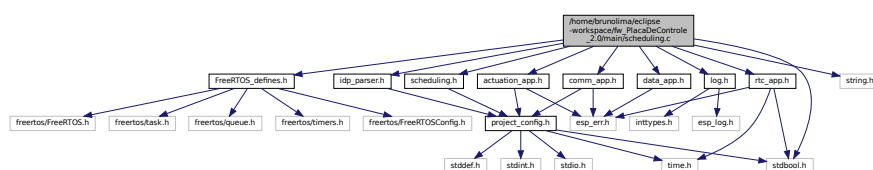
The application runs an infinite loop with a delay of 2000 milliseconds, effectively keeping the system_manager running.

7.33 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/scheduling.c File Reference

Implementation of the Scheduling module.

```
#include "scheduling.h"
#include "FreeRTOS_defines.h"
#include "log.h"
#include "actuation_app.h"
#include "data_app.h"
#include "comm_app.h"
#include "rtc_app.h"
#include "idp_parser.h"
#include <string.h>
#include <stdbool.h>
```

Include dependency graph for scheduling.c:



Macros

- #define **SCHEDULING_TAG** "scheduling"

Functions

- void [scheduling_start](#) ([idp_type](#) scheduling_idp, void *scheduling_data)
Starts a scheduling task based on the provided IDP and data.
- void [scheduling_register_callback](#) (const [app_callback](#) callback)
Registers a callback function for scheduling events.

7.33.1 Detailed Description

Implementation of the Scheduling module.

Author

soil-dev

Date

31 de jul. de 2023

7.33.2 Function Documentation

7.33.2.1 [scheduling_register_callback\(\)](#)

```
void scheduling_register_callback (
    const app\_callback callback )
```

Registers a callback function for scheduling events.

This function allows the registration of a callback function that will be invoked for scheduling events. The callback function is used to communicate with other modules or external systems.

Parameters

<i>callback</i>	The callback function to be registered.
-----------------	---

Note

The function checks for null values in the callback parameter.

7.33.2.2 scheduling_start()

```
void scheduling_start (
    idp_type scheduling_idp,
    void * scheduling_data )
```

Starts a scheduling task based on the provided IDP and data.

This function initiates a scheduling task based on the given IDP (InterDevice Protocol) and the associated data. The task is selected according to the IDP, and the data is used to configure and execute the task.

Parameters

<i>scheduling_idp</i>	The InterDevice Protocol identifier for scheduling.
<i>scheduling_data</i>	Pointer to the data structure associated with the scheduling task.

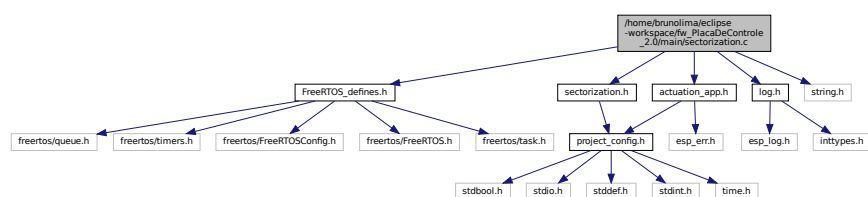
Note

The function checks for null values in the `scheduling_data` parameter and logs an error if found.

7.34 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/sectorization.c File Reference

Implementation file for sectorization functionality.

```
#include "sectorization.h"
#include "FreeRTOS_defines.h"
#include "log.h"
#include "actuation_app.h"
#include <string.h>
Include dependency graph for sectorization.c:
```



Macros

- `#define` **SECTORIZATION_TAG** "sectorization"

Functions

- void `sectorization_start` (sector_config sectors)
Starts the sectorization task with the provided sector configuration.
- void `sectorization_stop` (void)
Stops the sectorization task.

7.34.1 Detailed Description

Implementation file for sectorization functionality.

Author

soil-dev

Date

31 de jul. de 2023

7.34.2 Function Documentation

7.34.2.1 sectorization_start()

```
void sectorization_start (  
    sector_config sectors )
```

Starts the sectorization task with the provided sector configuration.

This function initializes the sectorization task with the specified sector configuration.

Parameters

<i>sectors</i>	Configuration for sectorization.
----------------	----------------------------------

7.34.2.2 sectorization_stop()

```
void sectorization_stop (  
    void )
```

Stops the sectorization task.

This function stops the sectorization task.

7.35 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_↵ 2.0/main/system_manager.c File Reference

Source file for the system manager module.

```
#include "system_manager.h"  
#include "project_config.h"
```


7.35.2 Function Documentation

7.35.2.1 system_manager_init()

```
void system_manager_init (
    void )
```

Initializes the system manager module.

Initializes the system manager.

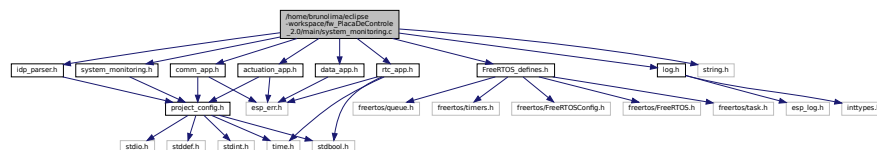
This function initializes various components of the system, including RTC, data storage, actuation, communication, sectorization, and scheduling. It also triggers an automatic reboot based on certain conditions.

7.36 /home/brunolima/eclipse-workspace/fw_PlacaDeControle_2.0/main/system_monitoring.c File Reference

System monitoring functionality implementation.

```
#include "system_monitoring.h"
#include "FreeRTOS_defines.h"
#include "log.h"
#include "actuation_app.h"
#include "data_app.h"
#include "comm_app.h"
#include "rtc_app.h"
#include "idp_parser.h"
#include <string.h>
```

Include dependency graph for system_monitoring.c:



Macros

- `#define SYSTEM_MONITORING_TAG "system monitoring"`
- `#define SYSTEM_DELAY_ANALYSIS_ANGLE_MS (6000)`

Enumerations

- enum `system_monitoring_states` { `SYSTEM_PAUSE` = 0 , `SYSTEM_RUNNING` , `SYSTEM_RETURN` }
- Enumeration representing the possible states of the system monitoring.*

Functions

- void [system_monitoring_start](#) (const pivot_return_config return_config, uint8_t monitoring_time)
Starts the system monitoring task with the provided configuration.
- void [system_monitoring_stop](#) (void)
Stops the system monitoring task.
- void [system_monitoring_register_callback](#) (const [app_callback](#) callback)
Registers a callback function for system monitoring.

7.36.1 Detailed Description

System monitoring functionality implementation.

Author

soil-dev

Date

4 de ago. de 2023

7.36.2 Enumeration Type Documentation

7.36.2.1 system_monitoring_states

enum [system_monitoring_states](#)

Enumeration representing the possible states of the system monitoring.

Enumerator

SYSTEM_PAUSE	System is in a paused state.
SYSTEM_RUNNING	System is running normally.
SYSTEM_RETURN	System is in a return state.

7.36.3 Function Documentation

7.36.3.1 system_monitoring_register_callback()

```
void system_monitoring_register_callback (  
    const app\_callback callback )
```


Registers a callback function for system monitoring.

This function allows the registration of a callback function to be used in the system monitoring module. The registered callback will be invoked by the system monitoring module to perform specific actions.

Parameters

in	<i>callback</i>	A function pointer to the callback function.
----	-----------------	--

7.36.3.2 system_monitoring_start()

```
void system_monitoring_start (
    const pivot_return_config return_config,
    uint8_t monitoring_time )
```

Starts the system monitoring task with the provided configuration.

Starts the system monitoring with the specified configuration.

This function initializes the system monitoring task with the specified configuration.

Parameters

<i>return_config</i>	Configuration for system monitoring.
<i>monitoring_time</i>	Time interval for system monitoring (in minutes).

7.36.3.3 system_monitoring_stop()

```
void system_monitoring_stop (
    void )
```

Stops the system monitoring task.

Stops the system monitoring.

This function stops the system monitoring task.

- actuation_app_manual_call
 - actuation_app.c, [23](#)
- actuation_app_set_actions
 - actuation_app.c, [23](#)
 - actuation_app.h, [34](#)
- actuation_app_set_config
 - actuation_app.c, [24](#)
 - actuation_app.h, [34](#)
- actuation_app_set_pump
 - actuation_app.c, [24](#)
 - actuation_app.h, [35](#)
- actuation_app_shutdown
 - actuation_app.c, [24](#)
 - actuation_app.h, [35](#)
- actuation_app_task
 - actuation_app.c, [24](#)
- actuator_read_percent
 - gpio_actuator.c, [52](#)
- actuator_wait_pressure
 - gpio_actuator.c, [52](#)
- app_callback
 - Project Configuration, [13](#)
- app_main
 - main.c, [126](#)
- arg_pair, [17](#)
 - type, [17](#)
 - value, [17](#)
- async_resp_arg, [18](#)
- bcd2dec
 - rtc_ds3231.c, [96](#)
 - rtc_ds3231.h, [89](#)
- CHECK_ARG
 - rtc_ds3231.c, [96](#)
- comm_app.c
 - comm_app_init, [26](#)
 - comm_app_send_idp_pack, [26](#)
 - comm_app_wifi_config, [27](#)
- comm_app.h
 - comm_app_init, [37](#)
 - comm_app_send_idp_pack, [38](#)
 - comm_app_wifi_config, [38](#)
- comm_app_init
 - comm_app.c, [26](#)
 - comm_app.h, [37](#)
- comm_app_send_idp_pack
 - comm_app.c, [26](#)
 - comm_app.h, [38](#)
- comm_app_wifi_config
 - comm_app.c, [27](#)
 - comm_app.h, [38](#)
- comm_type
 - Project Configuration, [13](#)
- CONFIG_HISTORY_MAX_VALUE
 - Project Configuration, [12](#)
- CONFIG_SCHEDULING_MAX_VALUE
 - Project Configuration, [12](#)
- CONFIG_SECTORS_MAX_VALUE
 - Project Configuration, [12](#)
- data_app.c
 - data_app_delete_scheduling, [29](#)
 - data_app_gen_scheduling_key, [29](#)
 - data_app_get_data_size, [30](#)
 - data_app_init, [30](#)
 - data_app_load, [30](#)
 - data_app_save, [31](#)
- data_app.h
 - data_app_delete_scheduling, [41](#)
 - data_app_gen_scheduling_key, [41](#)
 - data_app_get_data_size, [42](#)
 - data_app_init, [42](#)
 - data_app_load, [43](#)
 - data_app_save, [43](#)
 - DATA_TYPE_ACTIONS, [40](#)
 - DATA_TYPE_ECO_MODE_CONFIG, [40](#)
 - DATA_TYPE_HISTORY, [40](#)
 - DATA_TYPE_NETWORK_CONFIG, [40](#)
 - DATA_TYPE_OLD_HISTORY, [40](#)
 - DATA_TYPE_PIVOT_CONFIG, [40](#)
 - DATA_TYPE_RETURN_CONFIG, [40](#)
 - DATA_TYPE_SCHEDULING_ANGLE, [40](#)
 - DATA_TYPE_SCHEDULING_DATE, [40](#)
 - DATA_TYPE_SCHEDULING_OFF_ANGLE, [40](#)
 - DATA_TYPE_SCHEDULING_OFF_DATE, [40](#)
 - DATA_TYPE_SECTOR_CONFIG, [40](#)
 - data_type_t, [40](#)
 - DATA_TYPE_TIMESTAMP, [40](#)
- data_app_delete_scheduling
 - data_app.c, [29](#)
 - data_app.h, [41](#)
- data_app_gen_scheduling_key
 - data_app.c, [29](#)
 - data_app.h, [41](#)
- data_app_get_data_size
 - data_app.c, [30](#)
 - data_app.h, [42](#)
- data_app_init
 - data_app.c, [30](#)
 - data_app.h, [42](#)
- data_app_load
 - data_app.c, [30](#)
 - data_app.h, [43](#)
- data_app_save
 - data_app.c, [31](#)
 - data_app.h, [43](#)
- DATA_TYPE_ACTIONS
 - data_app.h, [40](#)
- DATA_TYPE_ECO_MODE_CONFIG
 - data_app.h, [40](#)
- DATA_TYPE_HISTORY
 - data_app.h, [40](#)
- DATA_TYPE_NETWORK_CONFIG
 - data_app.h, [40](#)
- DATA_TYPE_OLD_HISTORY
 - data_app.h, [40](#)
- DATA_TYPE_PIVOT_CONFIG

- data_app.h, [40](#)
- DATA_TYPE_RETURN_CONFIG
 - data_app.h, [40](#)
- DATA_TYPE_SCHEDULING_ANGLE
 - data_app.h, [40](#)
- DATA_TYPE_SCHEDULING_DATE
 - data_app.h, [40](#)
- DATA_TYPE_SCHEDULING_OFF_ANGLE
 - data_app.h, [40](#)
- DATA_TYPE_SCHEDULING_OFF_DATE
 - data_app.h, [40](#)
- DATA_TYPE_SECTOR_CONFIG
 - data_app.h, [40](#)
- data_type_t
 - data_app.h, [40](#)
- DATA_TYPE_TIMESTAMP
 - data_app.h, [40](#)
- dec2bcd
 - rtc_ds3231.c, [97](#)
 - rtc_ds3231.h, [90](#)
- ds3231_get_raw_temp
 - rtc_ds3231.c, [97](#)
 - rtc_ds3231.h, [91](#)
- ds3231_get_temp_float
 - rtc_ds3231.c, [98](#)
 - rtc_ds3231.h, [91](#)
- ds3231_get_temp_integer
 - rtc_ds3231.c, [98](#)
 - rtc_ds3231.h, [92](#)
- ds3231_get_time
 - rtc_ds3231.c, [99](#)
 - rtc_ds3231.h, [93](#)
- ds3231_init_desc
 - rtc_ds3231.c, [99](#)
 - rtc_ds3231.h, [93](#)
- ds3231_set_time
 - rtc_ds3231.c, [99](#)
 - rtc_ds3231.h, [94](#)
- eco_mode.c
 - eco_mode_register_callback, [115](#)
 - eco_mode_start, [115](#)
- eco_mode.h
 - eco_mode_register_callback, [117](#)
 - eco_mode_start, [117](#)
- eco_mode_register_callback
 - eco_mode.c, [115](#)
 - eco_mode.h, [117](#)
- eco_mode_start
 - eco_mode.c, [115](#)
 - eco_mode.h, [117](#)
- end_angle
 - pivot_sectors, [19](#)
- FreeRTOS Defines, [9](#)
- global_angle
 - Project Configuration, [15](#)
- GPIO_ACT_PIN_AUX
 - gpio_actuator.h, [58](#)
- GPIO_ACT_PIN_CCW
 - gpio_actuator.h, [58](#)
- GPIO_ACT_PIN_CCW_IN
 - gpio_actuator.h, [59](#)
- GPIO_ACT_PIN_CW
 - gpio_actuator.h, [59](#)
- GPIO_ACT_PIN_CW_IN
 - gpio_actuator.h, [59](#)
- GPIO_ACT_PIN_OFF
 - gpio_actuator.h, [59](#)
- GPIO_ACT_PIN_ON
 - gpio_actuator.h, [59](#)
- GPIO_ACT_PIN_PERC_AUX
 - gpio_actuator.h, [59](#)
- GPIO_ACT_PIN_PERC_IN
 - gpio_actuator.h, [60](#)
- GPIO_ACT_PIN_PERC_OUT
 - gpio_actuator.h, [60](#)
- GPIO_ACT_PIN_PRESS
 - gpio_actuator.h, [60](#)
- GPIO_ACT_PIN_PUMP
 - gpio_actuator.h, [60](#)
- GPIO_ACT_PIN_WATERING
 - gpio_actuator.h, [60](#)
- gpio_actuator.c
 - actuator_read_percent, [52](#)
 - actuator_wait_pressure, [52](#)
 - gpio_actuator_config, [53](#)
 - gpio_actuator_get, [53](#)
 - gpio_actuator_init, [53](#)
 - gpio_actuator_pressure_off, [54](#)
 - gpio_actuator_pressure_on, [54](#)
 - gpio_actuator_pump_off, [54](#)
 - gpio_actuator_pump_on, [54](#)
 - gpio_actuator_set, [54](#)
 - gpio_actuator_shutdown, [55](#)
 - gpio_actuator_start, [55](#)
 - vPercTimerOffExpire, [55](#)
 - vPercTimerOnExpire, [55](#)
- gpio_actuator.h
 - GPIO_ACT_PIN_AUX, [58](#)
 - GPIO_ACT_PIN_CCW, [58](#)
 - GPIO_ACT_PIN_CCW_IN, [59](#)
 - GPIO_ACT_PIN_CW, [59](#)
 - GPIO_ACT_PIN_CW_IN, [59](#)
 - GPIO_ACT_PIN_OFF, [59](#)
 - GPIO_ACT_PIN_ON, [59](#)
 - GPIO_ACT_PIN_PERC_AUX, [59](#)
 - GPIO_ACT_PIN_PERC_IN, [60](#)
 - GPIO_ACT_PIN_PERC_OUT, [60](#)
 - GPIO_ACT_PIN_PRESS, [60](#)
 - GPIO_ACT_PIN_PUMP, [60](#)
 - GPIO_ACT_PIN_WATERING, [60](#)
 - gpio_actuator_config, [61](#)
 - gpio_actuator_get, [62](#)
 - gpio_actuator_init, [62](#)
 - gpio_actuator_pressure_off, [62](#)

- gpio_actuator_pressure_on, 63
 - gpio_actuator_pump_off, 63
 - gpio_actuator_pump_on, 63
 - gpio_actuator_set, 63
 - gpio_actuator_shutdown, 64
 - GPIO_INPUT_PIN_GROUP, 60
 - GPIO_OUTPUT_PIN_GROUP, 61
- gpio_actuator_config
 - gpio_actuator.c, 53
 - gpio_actuator.h, 61
- gpio_actuator_get
 - gpio_actuator.c, 53
 - gpio_actuator.h, 62
- gpio_actuator_init
 - gpio_actuator.c, 53
 - gpio_actuator.h, 62
- gpio_actuator_pressure_off
 - gpio_actuator.c, 54
 - gpio_actuator.h, 62
- gpio_actuator_pressure_on
 - gpio_actuator.c, 54
 - gpio_actuator.h, 63
- gpio_actuator_pump_off
 - gpio_actuator.c, 54
 - gpio_actuator.h, 63
- gpio_actuator_pump_on
 - gpio_actuator.c, 54
 - gpio_actuator.h, 63
- gpio_actuator_set
 - gpio_actuator.c, 54
 - gpio_actuator.h, 63
- gpio_actuator_shutdown
 - gpio_actuator.c, 55
 - gpio_actuator.h, 64
- gpio_actuator_start
 - gpio_actuator.c, 55
- GPIO_INPUT_PIN_GROUP
 - gpio_actuator.h, 60
- GPIO_OUTPUT_PIN_GROUP
 - gpio_actuator.h, 61
- gprs_uart.h
 - gprs_uart_init, 66
 - gprs_uart_send_event, 66
- gprs_uart_init
 - gprs_uart.h, 66
- gprs_uart_send_event
 - gprs_uart.h, 66
- http_file_server_data, 18
- i2c_dev_read
 - rtc_i2c_dev.c, 101
- i2c_dev_t, 18
- i2c_dev_write
 - rtc_i2c_dev.c, 101
- i2c_master_init
 - rtc_i2c_dev.c, 102
- idp_parser.c
 - idp_parser_create_package, 68
 - idp_parser_create_pwd, 68
 - idp_parser_get, 69
 - idp_parser_get_packet_data, 69
 - idp_parser_get_pwd, 70
 - idp_parser_validate_actions, 70
 - idp_parser_validate_network, 70
- idp_parser.h
 - idp_parser_create_package, 72
 - idp_parser_create_pwd, 73
 - idp_parser_get, 73
 - idp_parser_get_packet_data, 74
 - idp_parser_get_pwd, 75
 - idp_parser_validate_actions, 75
 - idp_parser_validate_network, 76
- idp_parser_create_package
 - idp_parser.c, 68
 - idp_parser.h, 72
- idp_parser_create_pwd
 - idp_parser.c, 68
 - idp_parser.h, 73
- idp_parser_get
 - idp_parser.c, 69
 - idp_parser.h, 73
- idp_parser_get_packet_data
 - idp_parser.c, 69
 - idp_parser.h, 74
- idp_parser_get_pwd
 - idp_parser.c, 70
 - idp_parser.h, 75
- idp_parser_validate_actions
 - idp_parser.c, 70
 - idp_parser.h, 75
- idp_parser_validate_network
 - idp_parser.c, 70
 - idp_parser.h, 76
- idp_type
 - Project Configuration, 13
- log.h
 - LOG_ACTUATION, 106
 - LOG_COLOR_WHITE, 106
 - LOG_COMM, 107
 - LOG_DATA, 107
 - LOG_MANAGER, 107
- LOG_ACTUATION
 - log.h, 106
- LOG_COLOR_WHITE
 - log.h, 106
- LOG_COMM
 - log.h, 107
- LOG_DATA
 - log.h, 107
- LOG_MANAGER
 - log.h, 107
- main.c
 - app_main, 126
- nvs_data.c

- nvs_data_get_blob, 81
- nvs_data_get_size, 81
- nvs_data_init, 82
- nvs_data_set, 82
- nvs_data.h
 - nvs_data_get_blob, 78
 - nvs_data_get_size, 78
 - nvs_data_init, 79
 - nvs_data_set, 79
- nvs_data_get_blob
 - nvs_data.c, 81
 - nvs_data.h, 78
- nvs_data_get_size
 - nvs_data.c, 81
 - nvs_data.h, 78
- nvs_data_init
 - nvs_data.c, 82
 - nvs_data.h, 79
- nvs_data_set
 - nvs_data.c, 82
 - nvs_data.h, 79
- pivot_actions_off
 - actuation_app.c, 25
 - actuation_app.h, 36
- PIVOT_CCW
 - Project Configuration, 14
- PIVOT_CW
 - Project Configuration, 14
- PIVOT_DRY
 - Project Configuration, 14
- PIVOT_OFF
 - Project Configuration, 14
- PIVOT_ON
 - Project Configuration, 14
- PIVOT_PRESSURIZING
 - Project Configuration, 14
- pivot_sectors, 18
 - end_angle, 19
 - start_angle, 19
- pivot_states
 - Project Configuration, 13
- PIVOT_UNKNOWN
 - Project Configuration, 14
- PIVOT_WET
 - Project Configuration, 14
- Project Configuration, 11
 - __attribute__, 14
 - app_callback, 13
 - comm_type, 13
 - CONFIG_HISTORY_MAX_VALUE, 12
 - CONFIG_SCHEDULING_MAX_VALUE, 12
 - CONFIG_SECTORS_MAX_VALUE, 12
 - global_angle, 15
 - idp_type, 13
 - PIVOT_CCW, 14
 - PIVOT_CW, 14
 - PIVOT_DRY, 14
 - PIVOT_OFF, 14
 - PIVOT_ON, 14
 - PIVOT_PRESSURIZING, 14
 - pivot_states, 13
 - PIVOT_UNKNOWN, 14
 - PIVOT_WET, 14
- rf_uart.c
 - rf_uart_init, 86
 - rf_uart_send_event, 86
- rf_uart.h
 - rf_uart_init, 84
 - rf_uart_send_event, 84
- rf_uart_init
 - rf_uart.c, 86
 - rf_uart.h, 84
- rf_uart_send_event
 - rf_uart.c, 86
 - rf_uart.h, 84
- rtc_app.c
 - rtc_app_get_date_time, 49
 - rtc_app_get_str_date_time, 49
 - rtc_app_get_timestamp, 49
 - rtc_app_init, 50
 - rtc_app_set_timestamp, 50
 - rtc_show_date_time, 50
- rtc_app.h
 - rtc_app_get_date_time, 45
 - rtc_app_get_str_date_time, 45
 - rtc_app_get_timestamp, 46
 - rtc_app_init, 46
 - rtc_app_set_timestamp, 46
 - rtc_show_date_time, 47
- rtc_app_get_date_time
 - rtc_app.c, 49
 - rtc_app.h, 45
- rtc_app_get_str_date_time
 - rtc_app.c, 49
 - rtc_app.h, 45
- rtc_app_get_timestamp
 - rtc_app.c, 49
 - rtc_app.h, 46
- rtc_app_init
 - rtc_app.c, 50
 - rtc_app.h, 46
- rtc_app_set_timestamp
 - rtc_app.c, 50
 - rtc_app.h, 46
- rtc_ds3231.c
 - bcd2dec, 96
 - CHECK_ARG, 96
 - dec2bcd, 97
 - ds3231_get_raw_temp, 97
 - ds3231_get_temp_float, 98
 - ds3231_get_temp_integer, 98
 - ds3231_get_time, 99
 - ds3231_init_desc, 99
 - ds3231_set_time, 99
- rtc_ds3231.h
 - bcd2dec, 89

- dec2bcd, 90
- ds3231_get_raw_temp, 91
- ds3231_get_temp_float, 91
- ds3231_get_temp_integer, 92
- ds3231_get_time, 93
- ds3231_init_desc, 93
- ds3231_set_time, 94
- rtc_i2c_dev.c
 - i2c_dev_read, 101
 - i2c_dev_write, 101
 - i2c_master_init, 102
- rtc_i2c_dev_t, 19
- rtc_show_date_time
 - rtc_app.c, 50
 - rtc_app.h, 47
- scheduling.c
 - scheduling_register_callback, 127
 - scheduling_start, 127
- scheduling.h
 - scheduling_register_callback, 119
 - scheduling_start, 119
- scheduling_register_callback
 - scheduling.c, 127
 - scheduling.h, 119
- scheduling_start
 - scheduling.c, 127
 - scheduling.h, 119
- sectorization.c
 - sectorization_start, 129
 - sectorization_stop, 129
- sectorization.h
 - sectorization_start, 121
 - sectorization_stop, 121
- sectorization_start
 - sectorization.c, 129
 - sectorization.h, 121
- sectorization_stop
 - sectorization.c, 129
 - sectorization.h, 121
- start_angle
 - pivot_sectors, 19
- system_manager.c
 - system_manager_init, 131
- system_manager.h
 - system_manager_init, 122
- system_manager_init
 - system_manager.c, 131
 - system_manager.h, 122
- system_monitoring.c
 - system_monitoring_register_callback, 132
 - system_monitoring_start, 133
 - system_monitoring_states, 132
 - system_monitoring_stop, 133
 - SYSTEM_PAUSE, 132
 - SYSTEM_RETURN, 132
 - SYSTEM_RUNNING, 132
- system_monitoring.h
 - system_monitoring_register_callback, 124
 - system_monitoring_start, 124
 - system_monitoring_stop, 125
- system_monitoring_register_callback
 - system_monitoring.c, 132
 - system_monitoring.h, 124
- system_monitoring_start
 - system_monitoring.c, 133
 - system_monitoring.h, 124
- system_monitoring_states
 - system_monitoring.c, 132
- system_monitoring_stop
 - system_monitoring.c, 133
 - system_monitoring.h, 125
- SYSTEM_PAUSE
 - system_monitoring.c, 132
- SYSTEM_RETURN
 - system_monitoring.c, 132
- SYSTEM_RUNNING
 - system_monitoring.c, 132
- type
 - arg_pair, 17
- value
 - arg_pair, 17
- vPercTimerOffExpire
 - gpio_actuator.c, 55
- vPercTimerOnExpire
 - gpio_actuator.c, 55
- wifi_app.c
 - wifi_app_init, 113
 - wifi_app_set_config, 113
 - wifi_app_start, 114
- wifi_app.h
 - wifi_app_init, 111
 - wifi_app_set_config, 111
- wifi_app_init
 - wifi_app.c, 113
 - wifi_app.h, 111
- wifi_app_set_config
 - wifi_app.c, 113
 - wifi_app.h, 111
- wifi_app_start
 - wifi_app.c, 114